# pamplona.py, line by line

How the Alpha Rodeo Hidden Hand bot works: a study guide to read, annotate and explain

| | |
|---|---|
| **Team** | Alpha Rodeo |
| **Member** | Jaiv Patel (26075057) |
| **Mentor** | Arpit Gupta |
| **Final upload** | heist/professor.py = heist/pamplona.py (v14), 1,112 lines |
| **Event** | Hidden Hand, The Quant Club, IIT BHU |
| **Prepared** | Thursday 8 October 2026 |

Code colours: **keywords**, CONSTANTS, **function names**, numbers, "strings", # *comments*, *docstrings*. Grey numbers on the left are the line numbers of heist/pamplona.py.

# Contents

Chapters, then every code block of the walkthrough by its line numbers.

**How to use this guide** . . . 5

**The bot on one page** . . . 6

- The ten lines to remember . . . 6
- Key numbers . . . 6
- Live record of this file (entry #689) . . . 6

**1. The game in one page** . . . 8

- Cards and the stock . . . 8
- A deal has 5 rounds . . . 8
- Settlement and score . . . 8
- What the bot sees: the Obs object . . . 8
- The three methods the engine calls . . . 9
- A tiny example . . . 9

**2. The big picture** . . . 10

- The problem in one line . . . 10
- Job 1: inference (learning) . . . 10
- Job 2: decisions (planning) . . . 10
- Moves . . . 10
- Two safety layers . . . 10
- Where each part lives . . . 11

**3. Five ideas, with worked examples** . . . 12

- 3.1 The value of the stock given O (_ev, line 666) . . . 12
- 3.2 How the opponent values the stock (coefs, line 100) . . . 12
- 3.3 The 216 hypotheses (_init, line 428) . . . 12
- 3.4 Learning from an opponent quote: the Kalman filter (_see, lines 565–587) . . . 12
- 3.5 Learning from an answer to my quote (_see, lines 588–624) . . . 13
- 3.6 Weights and pruning (lines 587, 617, 461) . . . 13
- 3.7 Choosing a price (_quote, line 907) . . . 13
- 3.8 The utility (util, line 95) . . . 13
- 3.9 The swap (_move, line 1071) . . . 13

**4. Header, description and constants (lines 1–84)** . . . 15

- 1–4 The required header . . . 15
- 6–40 The description at the top . . . 15
- 42–43 Imports . . . 16
- 45–48 Basic numbers . . . 16
- 50–63 Opponent styles and their prior probabilities . . . 16

65–82 Noise, speed and decision settings . . . 17
83–84 Hypothesis fields . . . 18

**5. Maths helpers (lines 87–204) . . . 19**

87–92 The normal curve . . . 19
95–97 The utility . . . 19
100–116 The opponent's value as a straight line . . . 19
119–120 implied . . . 20
123–139 pick_q: which of my quotes a reader uses . . . 20
142–154 ekf: one Kalman step (the readable version) . . . 20
157–172 mix_into: hedging against outliers . . . 21
175–204 ekf_mix: the same, fused (the one actually used) . . . 21

**6. The house-reader attack (lines 207–415) . . . 23**

What this part is for . . . 23
207–221 Constants of the attack . . . 23
224–230 _lots: the sharp's trade size . . . 23
233–244 _Reader: its memory . . . 24
246–258 invert: their hand sum from one quote . . . 24
260–283 observe: is this quote consistent with the earlier ones? . . . 24
285–305 sync: feed new quotes and verify predictions . . . 25
307–310 active . . . 25
312–316 value: V when S is known . . . 26
318–327 their_base: their value for a given S and my last quote P . . . 26
329–340 events_left: who quotes next, until the end of the deal . . . 26
342–397 plan: exact look-ahead against the house bot . . . 26
399–415 quote and respond against an identified reader . . . 27

**7. The Bot: inference (lines 418–750) . . . 29**

418–425 Creating the bot . . . 29
427–459 _init: the prior and the 216 hypotheses . . . 29
461–465 _prune . . . 30
467–487 _state: the public numbers as the opponent saw them in round r . . . 30
489–501 _before: my position and Q before slot j . . . 31
503–511 _ks: the sign swap-expecting bots use . . . 31
513–529 _lastq: my last quote, carried forward . . . 31
531–548 _see, part 1: one observation . . . 31
549–564 _see, part 2: per-hypothesis set-up . . . 32
565–587 _see, part 3: learning from their quote . . . 32
588–624 _see, part 4: learning from their answer to my quote . . . 33
626–633 _see_swap: the opponent swapped . . . 34
635–664 _sync: process everything new, in order . . . 34
666–677 _ev: E[V | O] = α + β·O . . . 35
679–689 _opp_swap (not used any more) . . . 35

691–698 _mo: our best guess of O . . . 35
700–712 _p_reader . . . 35
714–750 _exact_reader: does the opponent fit the house reader formula? . . . 36

**8. The Bot: planning and decisions (lines 752–1112) . . . 37**

752–762 _slots: who quotes in each remaining slot . . . 37
764–780 _bel: their value under one hypothesis . . . 37
782–794 _roll: set-up . . . 37
795–823 _roll: my quotes . . . 38
824–838 _roll: their quotes . . . 38
840–852 _plan_setup . . . 39
854–872 _public (not used) and _plain_fair (the fallback) . . . 39
874–905 The three methods the engine calls . . . 39
907–924 _quote, part 1: set-up . . . 40
926–953 _quote, part 2: their answer model . . . 40
955–961 _quote, part 3: the immediate expected profit . . . 41
963–976 _quote, part 4: stage 1, the best price for this trade alone . . . 41
977–994 _quote, part 5: stage 2 candidates . . . 42
995–1019 _quote, part 6: score each candidate with the rest of the deal, then nudge . . . 42
1021–1037 _safe_quote . . . 43
1039–1069 _respond: buy, pass or sell? . . . 43
1071–1112 _move: never peek, swap only in round 5 . . . 44

**9. Every parameter in one table . . . 46**

**10. How we got here . . . 48**

Version history . . . 48
Ideas tested live and dropped . . . 48
How we tested . . . 48

**11. Questions the organisers may ask . . . 50**

**12. Glossary . . . 53**

**Notes . . . 54**

# How to use this guide

This guide explains **heist/pamplona.py** (v14), our final bot, line by line. heist/professor.py is the same code; only its line 4 is different. pamplona is marseille (v12) with two small changes to the round-5 swap: line 71 (swap whenever the expected gain is positive; marseille needed 4 points, changed in manila, v13) and line 1103 (decide on the average of the position we hold now and the position we expect to end with; pamplona, v14). Everything else is marseille's code.

Read it in this order:

- **The one-page summary and chapters 1–2** tell the whole story in a few pages. Read them twice.
- **Chapter 3** explains the five ideas you need (the value formula, the hypotheses, the Kalman filter, learning from answers, the planner). Its small examples were computed by running the real functions.
- **Chapters 4–8** are the line-by-line walkthrough. Each grey box is a piece of the file with its line numbers. Under it, every line or group of lines is explained ("L88" means line 88).
- **Chapters 9–12** list every parameter, the version history, questions the organisers may ask (with answers) and a glossary.

Two kinds of coloured boxes:

- **SAY IT LIKE THIS**: one or two sentences you can say out loud.
- **WHY**: why we chose it, and the test that decided it.

Every chapter ends with a box for your notes, and there are blank note pages at the end.

# The bot on one page

## The ten lines to remember

1. V = my hand + the board + **O**, the opponent's hand sum. O is the only real unknown (about 35 ± 8 at the start).
2. We keep **216 hypotheses** about how the opponent forms its price: naive, four kinds of reader, or blend quoter, times how honest its quotes are (lam), how it reacts to my trades (tau), whether it expects me to swap (kappa), and how much edge it needs to trade (2 or 7 points).
3. Each hypothesis has a Gaussian belief over (O, t), where t is how much the opponent trusts my quotes, plus a log-weight.
4. Their quote → **Kalman update** (under each hypothesis the quote is a straight line in O and t). Their answer to my quote → **truncated-normal update** (it tells us an interval). Their swap → the belief shifts and widens.
5. Weights add the log-probability each hypothesis gave to what happened (Bayes' rule). 10% of quotes and 6% of answers may be outliers. Hypotheses 6 log-units behind the best are dropped.
6. Value: **E[V | O] = α + β·O**.
7. Quote: stage 1 finds the best immediate profit (including **adverse selection**); stage 2 adds a play-out of the rest of the deal under the 6 likeliest hypotheses, through a square-root-like **utility** (like the score). Then the final quote is **70% planner + 30% safe quote** (their last quote, 1.3 towards our value).
8. Answer: buy, pass or sell, compared with the same yardstick.
9. Move: **never peek**. Swap at most once, **only in round 5**: our lowest card if we expect to end long, our highest if short, whenever the expected gain is positive, judged on the average of our current and our expected final position.
10. An exact attack on the house bots (off against real teams), try/except fallbacks everywhere, nothing kept between deals, standard library only (heapq, math).

## Key numbers

| Thing | Value | Where |
|---|---|---|
| hypotheses | 216 | _init, line 428 |
| hypotheses used to plan | 6 (NPLAN) | line 73 |
| pruning distance | 6 log-units = 403× | line 70 |
| utility curvature | RISK = 60 | line 78 |
| nudge | 30% (SAFE_W), step 1.3 (SAFE_D) | lines 79–80 |
| prior: does not read my quotes | 30% (P_NAIVE) | line 51 |
| opponent take threshold | 2 points (75%) or 7 (25%) | line 60 |
| swap bar in round 5 | 0: any positive expected gain (marseille: 4) | line 71 |
| swap decided on | average of current and expected final position | line 1103 |
| speed (local) | median 0.20 ms, 99% under 1 ms per call | ledger |

## Live record of this file (entry #689)

Paired against its parent manila on the same 24 opponents: +0.40 ± 0.13, better against 23 of them, including PaperHands (+0.84), Imposters (+1.24), TeenPatti (+1.19), Genz7 (+0.34) and HAN SOLO (+0.30). Before it, manila (entry #665) beat marseille by +0.10 ± 0.05 and marseille (#636) matched bogota (+0.02 ± 0.07). The last

test, rr02 (#701), did not beat pamplona (−0.11 ± 0.27), so pamplona is the final.

| Your notes |
| --- |
| |
| |
| |
| |

# 1. The game in one page

## Cards and the stock

- One 52-card deck. A card is worth its rank: A = 1, 2 to 10, J = 11, Q = 12, K = 13. There are four copies of every rank, so the whole deck adds up to 4 × 91 = **364** (you will see 364 in the code).
- Each player gets 5 private cards, the **hand**. Five more cards are the face-down **board**. The other 37 cards are the **stack**, used only for swaps.
- The stock settles at **V = my hand + the opponent's hand + the whole board**: 15 cards, 105 on average.

## A deal has 5 rounds

- At the start of round r, board card r − 1 is turned face up: round 1 shows none, round 5 shows four. The fifth board card is never shown before settlement.
- Each round starts with a **move**, chosen by both bots at the same time: nothing, **PEEK** (pay 3, see one random card of the opponent's hand) or **SWAP** (free, once per deal: throw one card face up, draw a fresh one from the stack).
- Then two **quotes**. The **maker** names one price p. The **taker** answers: buy up to 2 lots at p + 2, sell up to 2 lots at p − 2, or pass. Then the roles swap.
- Nobody may hold more than **10 lots** long or short; the engine cuts any trade that would break this. A maker that fails to quote is quoted at 105.

| Round | Board shown | Move phase | First maker | Second maker |
|---|---|---|---|---|
| 1 | 0 cards | both choose | seat 0 | seat 1 |
| 2 | 1 card | both choose | seat 1 | seat 0 |
| 3 | 2 cards | both choose | seat 0 | seat 1 |
| 4 | 3 cards | both choose | seat 1 | seat 0 |
| 5 | 4 cards | our swap happens here | seat 0 | seat 1 |

So there are 10 quotes per deal, the "slots" 0 to 9 (slot s is in round s // 2 + 1). Each bot quotes 5 times and answers 5 times.

## Settlement and score

- PnL = cash from trades + position × V − money spent on peeks.
- Every deal is played twice with the seats swapped (same cards), so the luck of the cards cancels. For each pair, p = the PnL over both plays; the score is the average of **sign(p) × √|p|**. Winning by 100 counts 10, winning by 25 counts 5, so many steady wins beat one huge win.
- A call that takes more than 50 ms is replaced by a fallback answer. Our target is far below that.

## What the bot sees: the Obs object

| Field | Meaning |
|---|---|
| round | 1 to 5 |
| hand | my 5 cards now (ranks 1 to 13) |
| board | the board cards face up so far |
| opp_known | opponent cards I peeked that they still hold (always empty for us: we never peek) |
| unseen | unseen[r − 1] = copies of rank r I have not seen anywhere (not in my hand, the board, the discards or my peeks) |
| position | lots I hold: + long, − short |

| Field | Meaning |
|---|---|
| cash, spent | money from my trades; money paid for peeks |
| tape | every quote so far, oldest first: (round, maker "me" or "opp", price, qty). qty is what the TAKER did: +n = bought n at price + 2, −n = sold n at price − 2, 0 = passed |
| discards | every swapped-out card: (round, who, rank) |
| peeks | every peek: (round, who); nobody learns which card was seen |

## The three methods the engine calls

- `move(obs)` returns None, "PEEK" or ("SWAP", i).
- `quote(obs)` returns a price (the engine clamps it to 0..195 and rounds to 0.01).
- `respond(obs, price)` returns −2 to +2 lots.

## A tiny example

I quote 100. The opponent buys 2 lots: it pays 102 each and I am now short 2. The final V is 110. My PnL = 2 × (102 − 110) = −16. The opponent bought because it knew its hand was high. Losing to the better-informed side like this is called **adverse selection**, and much of the code is about avoiding it.

**Your notes**

# 2. The big picture

## The problem in one line

V = my hand (known) + the board (partly known) + **O**, the opponent's hand sum, which is the only real unknown. So the game is two things: (1) estimate O, and (2) know what the opponent believes, because that decides how it trades against my prices.

## Job 1: inference (learning)

We keep a list of 216 **hypotheses**, each a possible opponent style: does it read my quotes, how honest are its own quotes, how much edge does it need before it trades, and so on. Each hypothesis carries its own Gaussian belief about (O, t) (t = how much the opponent trusts my quotes) and a weight that says how well it has predicted the opponent so far. Every opponent quote and every answer to my quotes updates all of them.

## Job 2: decisions (planning)

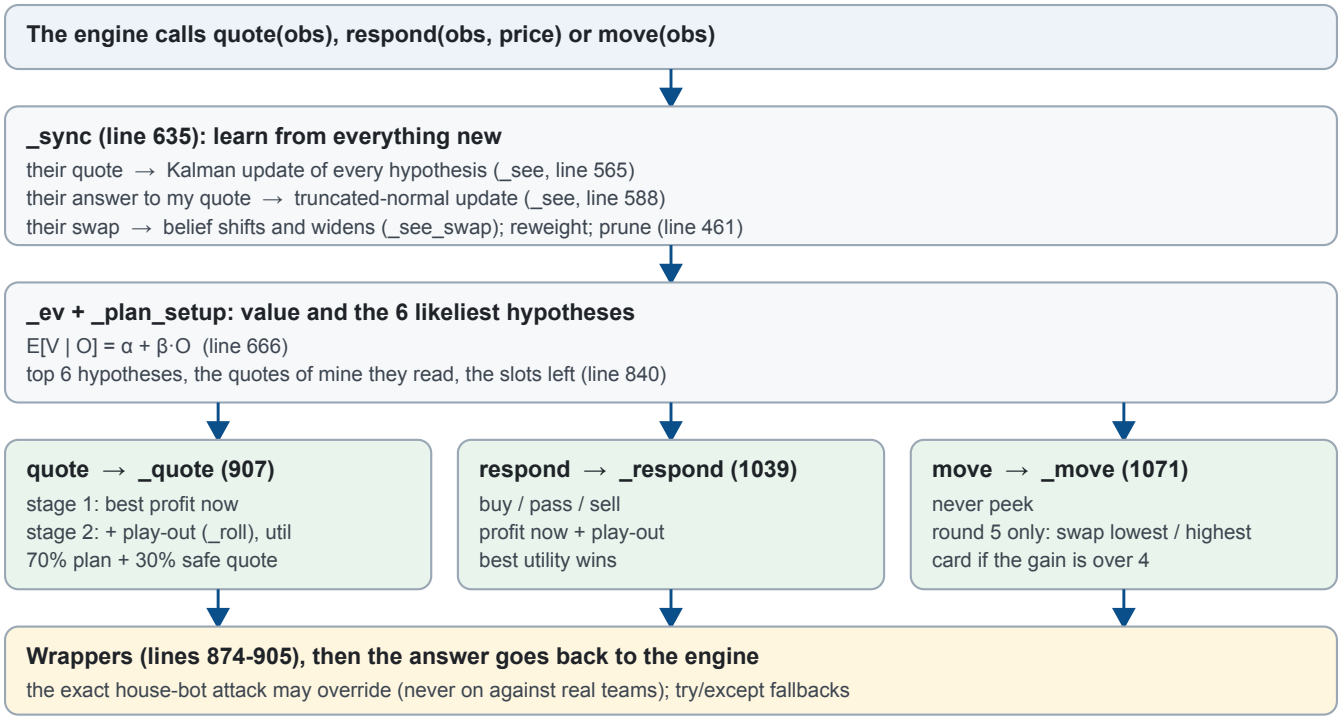
For each possible action (a price to quote; buy, pass or sell) we compute the expected profit now plus a quick simulation of the rest of the deal under the 6 most likely hypotheses, score the total with a concave utility that mirrors the √ in the score, and take the best. Our final quote is then moved 30% of the way towards a **safe quote**: the opponent's last quote, moved 1.3 towards our value.

## Moves

We never peek. We swap at most once, only in round 5: our lowest card if we expect to end long, our highest if we expect to end short, whenever the expected gain is positive. The decision uses the average of the position we hold now and the position we expect to end with.

## Two safety layers

- An exact attack on the house bots "reader" and "sharp" (class _Reader). It switches on only when the opponent's quotes match the house formula to the cent, and off at the first surprise. Against real teams it stays off.
- Every engine call is wrapped in try/except with a simple fallback, so a bug can never crash a deal.



## Where each part lives

| Lines | Part |
|---|---|
| 1–84 | header, description, constants |
| 87–204 | maths helpers: normal curve, utility, the opponent's value formula, Kalman update |
| 207–415 | _Reader: exact attack on the house reader and sharp bots |
| 418–465 | Bot: creation, the 216 hypotheses, pruning |
| 467–750 | Bot: inference (public state, learning from quotes, answers and swaps) |
| 752–852 | Bot: planning helpers (slots, beliefs, play-outs) |
| 854–905 | the three engine methods with their fallbacks |
| 907–1019 | _quote: choosing a price |
| 1021–1037 | _safe_quote: the nudge target |
| 1039–1069 | _respond: buy, pass or sell |
| 1071–1112 | _move: the round-5 swap |

**SAY IT LIKE THIS**

"Our bot is a Bayesian trader. It keeps 216 guesses about the opponent's style, each with a belief about the opponent's hand that a Kalman filter updates from every quote and answer. To act, it scores each option by profit now plus a play-out of the rest of the deal under the six likeliest guesses, through a utility shaped like the score, and then hides a little by moving its quote 30% towards the opponent's last quote."

**Your notes**

# 3. Five ideas, with worked examples

All numbers in this chapter come from running the functions in pamplona.py.

## 3.1 The value of the stock given O (_ev, line 666)

The unknowns for me are the opponent's 5 cards (sum O) and the hidden board cards. If I knew O, the hidden board cards would be random cards from the rest of my unseen pool. So

**E[V | O] = α + β × O**, with α = my hand + visible board + (hidden board cards) × (unseen sum) / (unseen count − 5) and β = 1 − (hidden board cards) / (unseen count − 5).

Example, round 1, my hand sum 40: there are 47 unseen cards worth 364 − 40 = 324. α = 40 + 5 × 324 / 42 = 78.57 and β = 0.881. If O = 35 then V ≈ 109.4; if O = 45 then V ≈ 118.2.

Why β is below 1: if the opponent's hand is high, the cards left for the board are a little lower on average.

## 3.2 How the opponent values the stock (coefs, line 100)

From the opponent's side, my hand is the unknown. A **naive** opponent uses only its own hand and the public cards. In round 1 its fair value is 77.45 + 0.787 × O (105.0 if O = 35, 112.9 if O = 45).

A **reader** also reads my last quote q. It works out the hand my quote implies (function implied) and moves its fair value by t × (implied hand − average hand), where t is its trust in me. Example: my quote 115 implies a hand of 45, ten above average; a reader with t = 0.6 and O = 35 moves from 105 to 111.

So under every hypothesis, the opponent's fair value is a straight line in O and t:

**fair = A0 + A1·t + (B0 + B1·t)·O** (plus two small extra terms, tau·Q and kappa·ks).

That straight-line form is what makes the Kalman filter work.

## 3.3 The 216 hypotheses (_init, line 428)

| Family | Rule | Prior share | Count |
|---|---|---|---|
| naive | does not read my quotes | 0.30 × 0.88 | 24 |
| reader RAW | reads my last quote as it is | 0.70 × 0.55 × 0.88 | 24 |
| reader CLAMP | caps the hand my quote implies to 5..65 | 0.70 × 0.20 × 0.88 | 24 |
| reader SANE | ignores quotes implying a hand outside 12..58 | 0.70 × 0.08 × 0.88 | 24 |
| reader CUR | also reads the price it is answering | 0.70 × 0.17 × 0.88 | 24 |
| blend (BLD) | quotes a mix of its own value and my last quote | 0.12 | 96 |

Inside each family the hypotheses differ in:

- **lam**: how much of its own hand its quotes show (1, 0.7, 0.4 or 0; 0 means its quotes say nothing).
- **tau**: how much its value moves per lot I buy from it (0, 1.5 or 3; only for fully honest quoters).
- **kappa**: 0 or 3.9, its expectation that I will swap away a bad card.
- **threshold**: it needs 2 or 7 points of edge before it trades.
- blend quoters also have a **skew**: 0 or 0.9 per lot of my position.

Each hypothesis is a Python list of 13 numbers (names H_READS … H_TH, line 84): its fixed style, its belief over (O, t) as two means and a 2×2 covariance, and its log-weight.

## 3.4 Learning from an opponent quote: the Kalman filter (_see, lines 565–587)

Under a hypothesis, the opponent's quote is a straight line in (O, t) plus a little noise (SIG_Q = 0.5). For a Gaussian belief and a linear noisy reading, the **Kalman filter** is the exact Bayes update: new mean = old mean + gain × (quote − predicted quote), and the variance shrinks.

Example: an honest naive hypothesis in round 1 with belief O ~ N(35, $8^2$). It predicts the quote 105.0. The opponent quotes 112.87, a surprise of 7.87. The gain is 1.262, so a pure Kalman step gives O ≈ 44.9 with sd 0.6: one honest quote almost reveals the hand.

We also allow a 10% chance that a quote is unexplained (EPS_Q). Here the quote is plausible, so it counts as explained with probability 0.929, and the belief becomes O ≈ 44.2 with sd 3.4. The hypothesis' log-weight changes by log(0.0281) = −3.57; hypotheses that predicted the quote better lose less weight.

## 3.5 Learning from an answer to my quote (_see, lines 588–624)

When I quote p and an opponent with take threshold th **passes**, its fair value must be between p − th and p + th. When it **buys**, its fair value is above p + th; when it **sells**, below p − th. That is an interval, not a number, so the update uses the mean and variance of a normal distribution cut to that interval (a **truncated normal**), followed by the same Kalman step.

Example (same hypothesis, I quote 100): before the answer, their fair value is believed to be 105.0 ± 6.4.

- They **pass** (fair value in 98..102): this was only 18% likely. O moves from 35 to 29.6 and its sd from 8 to 3.6.
- They **buy** (fair value above 102): 68% likely. O moves to 39.0, sd 5.7.

## 3.6 Weights and pruning (lines 587, 617, 461)

Each hypothesis' log-weight adds log(the probability it gave to what happened). This is Bayes' rule in log form. After every event, hypotheses more than 6 log-units behind the best ($e^6$ = 403 times less likely) are dropped (PRUNE), and the best is reset to 0 so the numbers never underflow.

## 3.7 Choosing a price (_quote, line 907)

- **1. Their answer model.** Under each of the 6 likeliest hypotheses, the opponent's fair value when answering me is normal with mean f and sd sf.
- **2. Immediate expected profit** of quoting p (function imm): they buy nb lots with probability Φ((f − p − th) / sf), and I sell at p + 2. But they buy exactly when their hand is high, which is when V is high, so the formula subtracts β × c × φ(z), the **adverse-selection** term (c = how much O moves with their fair value).
- **3. Stage 1:** try their fair value ± 2, 2.5, 3, 3.5, 4.5 and 6 and keep the price with the best immediate profit.
- **4. Stage 2:** for a short list of candidates (that price, their fair value, ± 3.05, ± 6.05, and extreme prices at the position limit), add a play-out of the rest of the deal (function _roll) and score the whole deal with the utility. Keep the best.
- **5. Nudge:** final quote = 0.7 × best + 0.3 × safe quote.

## 3.8 The utility (util, line 95)

util(x) = x / √(1 + |x| / 60). Small amounts count fully (util(10) = 9.3); large amounts are shrunk like a square root (util(100) = 61.2, util(400) = 144.5). The score uses sign(p)√|p| per deal pair, so we choose actions by the same shape: a sure +30 is worth more to us than a coin flip between 0 and +60, because util(30) = 24.5 is more than (0 + util(60)) / 2 = 21.2.

## 3.9 The swap (_move, line 1071)

In round 5, before the last two quotes, let pos be the position we hold now and fpos the position we expect to end with (from play-outs of round 5). pamplona uses x = (pos + fpos) / 2. If x > 0 (we will be long), swapping our lowest card c for a random stack card (worth mstack on average) changes our PnL by about x × (mstack − c); if x < 0 we throw our highest card instead. We swap whenever that expected gain is positive. Example: pos = +4 and fpos = +8, so x = +6; lowest card 2, mstack 7: gain 6 × 5 = 30, which is positive, so we swap. Why the average: the play-out tends to predict more round-5 trading than really happens, and averaging it with the real position made the decision steadier (+0.40 live).

**Your notes**

# 4. Header, description and constants (lines 1–84)

This chapter covers the top of the file: the required header, the description, the imports and every constant.

### Lines 1–4 · The required header

```
1  # Team Name: Alpha Rodeo
2  # Members: Jaiv Patel(26075057)
3  # Mentor: Arpit Gupta
4  # Version: pamplona (v14), parent manila, idea: round-5 swap decided by the average of my current and
       my expected final position (was the expected final position alone); field2 +0.115 +-0.016 / +0.129
       +-0.018 on two seed pairs; live #689 +0.40 +-0.13 vs manila (24 shared opponents, better on 23)
```

**L1–3** The three lines every submission must start with, exactly as the rules give them: team name, member, mentor.

**L4** Our version line: its name (pamplona, v14 in our ledger), its parent (manila) and the one idea it adds: the round-5 swap is decided on the average of the current and the expected final position. It also records the evidence: two local seed pairs (+0.115, +0.129) and the live result (+0.40 ± 0.13 against manila, better on 23 of 24 opponents). (manila added the swap bar 0; marseille was bogota made 30–40% faster.)

> **SAY IT LIKE THIS**
> "Lines 1–3 are the required header. Line 4 says which version this is and what it changed."

### Lines 6–40 · The description at the top

```
6  """
7  rival_2.py - Bayesian opponent model + a planner that exploits readers.
8
9  The stock is worth my hand + the board + the opponent's hand 0, so the game is
10 estimating 0, and knowing how the opponent estimates mine.
11
12 Inference
13     A set of hypotheses about HOW the opponent forms its fair value:
14         reads   does it read my quotes at all (naive vs reader)
15         rule    raw / clamped to a legal hand / ignores implausible quotes /
16                 also reads the very price it is answering
17         lam     how much of its own hand its quotes reveal (1 honest .. 0 none;
18                 0 also covers strategic quoters, whose quotes mean nothing)
19         tau     how far it moves its estimate of my hand per lot I trade
20         kappa   bias for "the opponent will swap" (ai-style bots)
21     Each hypothesis carries a 2-D Gaussian over (0, trust t) updated by an
22     extended Kalman filter: their quotes are noisy linear readings of their
23     fair value; their responses to my quotes say their fair value is above /
24     below / within 2 of my price (truncated-normal update). Hypotheses are
25     reweighted by how well they predicted what happened; outliers are expected.
26
27 Decisions
28     Every candidate action (a price to quote; buy / pass / sell) is scored as
29     the expected profit now plus a play-out of the rest of the deal under the
30     top hypotheses, through a concave (square-root-like) utility, as the score.
31     Readers believe my quotes, which makes them steerable:
32       * at +-10 lots my quote cannot be traded against in the bad direction,
33         so an extreme price is free. A reader then thinks the stock is worth
34         ~170 (or ~20) and trades into my position at absurd prices.
35       * a paid "probe" (a quote implying a hand of 63 or 7) is only used once
36         the opponent is clearly a reader with informative quotes, and only
37         when its next quote will show whether the probe worked.
38     Swap: lowest card if I expect to end long, highest if short, when the
39     expected final position x (new card - old card) is large enough.
40 """
```

The text between the triple quotes is the module docstring: a summary of the whole bot. Python ignores it when running.

**L7** "rival_2.py" was the name of the first file of this family; we kept its description. Our versions since: tokyo, berlin, … bogota, marseille, manila, pamplona.

**L9–10** The key sentence: the stock is my hand + the board + O, so the game is estimating O, and knowing how the opponent estimates my hand.

**L12–25** Inference: what each part of a hypothesis means (reads, rule, lam, tau, kappa), the 2-D Gaussian over (O, t), the Kalman update for quotes, the truncated-normal update for answers, reweighting, and the allowance for outliers.

**L27–37** Decisions: every action is scored by profit now plus a play-out, through a concave utility. "Readers are steerable" describes the free extreme quote at ±10 lots and the paid probes. In pamplona (as in marseille) the paid probes are switched off (P_EXT = P_PROBE = 2.0, lines 74 and 76); only the free extreme quote at the limit is still used.

**L38–39** The swap rule.

## Lines 42–43 · Imports

```
42 import heapq
43 import math
```

**L42–43** Only `heapq` (to pick the 6 heaviest hypotheses quickly) and `math` (sqrt, exp, log, erf). Both are on the allowed list. There is no file or network access anywhere in the bot.

## Lines 45–48 · Basic numbers

```
45 MAXP = 195.0
46 SQ2 = math.sqrt(2.0)
47 ISQ2PI = 1.0 / math.sqrt(2.0 * math.pi)
48 INF = float("inf")
```

**L45** MAXP = 195 = 13 × 15, the highest possible V. The engine clamps every quote to 0..195.

**L46** √2, used in Phi.

**L47** 1 / √(2π), the constant in the normal density.

**L48** Infinity, used for one-sided intervals such as "their fair value is above 102".

## Lines 50–63 · Opponent styles and their prior probabilities

```
50 RAW, CLAMP, SANE, CUR, BLD = 0, 1, 2, 3, 4
51 P_NAIVE = 0.3
52 P_BLEND = 0.12      # prior mass of blend quoters (the rest scaled by 1 − P_BLEND)
53 BLEND_LAM = ((1.0, 0.5), (0.7, 0.2), (0.4, 0.15), (0.2, 0.15))
54 BLEND_SK = ((0.0, 0.5), (0.9, 0.5))   # their quote skew per lot of MY position
55 W_MEAN, W_SD = 0.45, 0.25
56 BLEND_TAU = ((0.0, 0.3), (1.5, 0.4), (3.0, 0.3))
57 CARRY = 0.82      # how much a board card moves a carried fair, per point off 7
58 RULE_PRIOR = {RAW: 0.55, CLAMP: 0.2, SANE: 0.08, CUR: 0.17}
59 LAM_PRIOR = ((1.0, 0.45), (0.7, 0.12), (0.4, 0.08), (0.0, 0.35))
60 TH_PRIOR = ((2.0, 0.75), (7.0, 0.25))   # their take threshold: |fair − my price| (2 values for speed)
61 TAU_PRIOR = ((0.0, 0.6), (1.5, 0.25), (3.0, 0.15))      # only for honest quoters
62 KAP_PRIOR = ((0.0, 0.8), (3.9, 0.2))
63 T_MEAN, T_SD = 0.6, 0.3
```

**L50** Five rule codes: RAW, CLAMP, SANE, CUR (the readers) and BLD (blend quoters). Small integers are faster to compare than strings.

**L51** P_NAIVE = 0.3: prior chance that the opponent does not read my quotes at all.

**L52** P_BLEND = 0.12: prior chance that it is a blend quoter. All other families share the remaining 0.88.

**L53** BLEND_LAM: for blend quoters, how much of their hand shows in their own value: 1.0 (prob. 0.5), 0.7 (0.2), 0.4 (0.15), 0.2 (0.15).

**L54** BLEND_SK: their quote moves 0 or 0.9 per lot of MY position (an inventory skew), 50/50.

**L55** W_MEAN, W_SD: prior for the blend weight W that a blend quoter puts on my last quote: mean 0.45, sd 0.25.

**L56** BLEND_TAU: for blend quoters, how much their value moves per lot I buy from them: 0, 1.5 or 3.

**L57** CARRY = 0.82: when a blend quoter carries my old quote forward, each new board card moves it by 0.82 per point away from 7. A shown card changes the value of the stock by a bit less than its distance

from 7, because the average of the cards still unknown moves the other way: about 1 − 9/46 = 0.80 in round 2 and 0.86 in round 5.

**L58** RULE_PRIOR: among readers, RAW 55%, CLAMP 20%, SANE 8%, CUR 17%.

**L59** LAM_PRIOR: how honest their quotes are: lam 1 (45%), 0.7 (12%), 0.4 (8%) or 0 (35%: quotes say nothing about the hand, as with strategic quoters and "hiders").

**L60** TH_PRIOR: their take threshold: they trade on 2 points of edge (75%) or only on 7 (25%). Many strong teams only trade on large edges.

**L61** TAU_PRIOR: 0 (60%), 1.5 (25%), 3 (15%); used only for lam = 1 quoters.

**L62** KAP_PRIOR: 80% "does not expect me to swap" (0); 20% "moves its value by 3.9 in the direction my position says I will swap".

**L63** T_MEAN, T_SD: prior trust of a reader in my quotes: mean 0.6, sd 0.3.

**WHY / EVIDENCE**

TH_PRIOR came in with helsinki (v06): modelling opponents who need 7 points of edge was worth +0.86 on our local field model. Taking the thresholds out again (test z02) lost −0.45 live, and moving the prior to 60/40 (test mm03, today 18:10) lost −0.53 ± 0.32 live, so 75/25 stays.

## Lines 65–82 · Noise, speed and decision settings

```
65  SIG_Q = 0.5       # quote model noise
66  EPS_Q = 0.1       # chance a quote is unexplained
67  U_Q = 0.02        # density of an unexplained quote
68  SIG_R = 1.0       # response threshold noise
69  EPS_R = 0.06      # chance a response is unexplained
70  PRUNE = 6.0       # drop hypotheses this many log-units behind the best
71  SWAP_TH = (15.0, 12.0, 9.0, 6.0, 0.0)   # by round: waiting has option value
72  DELTAS = (2.0, 2.5, 3.0, 3.5, 4.5, 6.0)
73  NPLAN = 6         # hypotheses used in play-outs
74  P_EXT = 2.0       # confidence in a raw reader needed to PAY for an extreme quote
75  EX_TOL = 0.02      # exact-reader test: every quote within this of the house reader formula
76  P_PROBE = 2.0    # confidence in any reader needed to pay for a probe (implied 63 / 7)
77  LAM_PROBE = 0.4   # ... and only if its quotes are informative (strategic quoters are not)
78  RISK = 60.0       # utility curvature
79  SAFE_D = 1.3      # safe quote: this far from their last quote, towards my value
80  SAFE_W = 0.3      # share of the safe quote in my quote
81  OPP_SWAP_K = 0.6  # share of the opponent's coming swap priced into my value
82
```

**L65** SIG_Q = 0.5: how far (sd) a quote can be from what a correct hypothesis predicts.

**L66–67** EPS_Q = 0.1 and U_Q = 0.02: 10% of quotes may be "unexplained"; such a quote gets a flat probability density of 0.02 (as if spread evenly over about 50 points). One odd quote then cannot destroy a good hypothesis.

**L68** SIG_R = 1.0: noise on the take threshold when we read answers.

**L69** EPS_R = 0.06: 6% of answers may be random (each of buy, pass and sell then has 2%).

**L70** PRUNE = 6.0: drop hypotheses 6 log-units (403 times) less likely than the best. It was 12 in bogota; 6 makes calls about 40% faster and changed only about 5% of decisions in our check (local score +0.014).

**L71** SWAP_TH: the gain needed to swap, by round: 15, 12, 9, 6, 0. We only swap in round 5, so only the last value is used, and it is 0: we swap whenever the expected gain is positive. manila (v13) changed it from marseille's 4 to 0, and pamplona keeps it.

**L72** DELTAS: how far from their fair value the stage-1 price candidates are: 2, 2.5, 3, 3.5, 4.5, 6.

**L73** NPLAN = 6: number of most likely hypotheses used in decisions (a speed versus accuracy choice).

**L74** P_EXT = 2.0: the confidence needed to PAY for an extreme quote. A probability is never above 1, so 2.0 switches paid extremes off.

**L75** EX_TOL = 0.02: the house-reader test: every quote must be within 0.02 of the house formula.

**L76** P_PROBE = 2.0: the same trick: paid probes are off.

**L77** LAM_PROBE = 0.4: for probes (now off), the reader would also have to show its hand in its quotes.

**L78** RISK = 60: the curvature of the utility (section 3.8).

**L79** SAFE_D = 1.3: the safe quote is 1.3 from their last quote, towards my value, inside their ±2 no-trade band.

**L80** SAFE_W = 0.3: the nudge: 30% safe quote, 70% planner.

**L81** OPP_SWAP_K = 0.6: belongs to _opp_swap (line 680), which is no longer called (since palermo).

**WHY / EVIDENCE**

The swap bar: lowering it improved the local score step by step (average of two seed pairs: bar 6 −0.060, 4 0, 2.5 +0.026, 1 +0.054, 0 +0.083). Live, manila (bar 0) beat marseille (bar 4) by +0.10 ± 0.05 on the same 37 opponents. Round 5 is the last chance to swap, so waiting has no value left.

**WHY / EVIDENCE**

Paid probes were switched off in stockholm (v09): in a local test one probe quote of 146.18 cost 100 points in a single deal, and turning probes off was neutral to positive. RISK 30 and 120 were tested: 120 lost (−0.08), 30 was not clearly better.

## Lines 83–84 · Hypothesis fields

```
83  # hypothesis fields
84  H_READS, H_RULE, H_LAM, H_TAU, H_KAP, H_MO, H_MT, H_P00, H_P01, H_P11, H_LW, H_SK, H_TH = range(13)
```

**L84** A hypothesis is a list of 13 numbers; these names are the positions in the list:

- H_READS (does it read my quotes?), H_RULE (RAW, CLAMP, SANE, CUR or BLD), H_LAM (honesty of its quotes), H_TAU (move per lot I buy), H_KAP (swap expectation), H_SK (blend skew), H_TH (take threshold). These are fixed for the hypothesis.
- H_MO and H_MT: the belief means of O and t. H_P00, H_P01, H_P11: the belief covariance (variance of O, covariance of O and t, variance of t). These are updated by learning.
- H_LW: the log-weight.
- Lists with index constants are faster in Python than objects with attributes, which matters for the time limit.

**Your notes**

# 5. Maths helpers (lines 87–204)

Small functions used everywhere: the normal curve, the utility, the opponent's value formula, and the Kalman update.

### Lines 87–92 · The normal curve

```
87  def Phi(x):
88      return 0.5 * (1.0 + math.erf(x / SQ2))
89
90
91  def pdf(x):
92      return math.exp(-0.5 * x * x) * ISQ2PI
```

**L87–88** Phi(x): the standard normal cumulative probability P(Z ≤ x), computed with math.erf.

**L91–92** pdf(x): the standard normal density φ(x) = $e^{-x^2/2}$ / √(2π).

### Lines 95–97 · The utility

```
95  def util(x):
96      """Concave utility: linear for small PnL, square-root-like for large."""
97      return x / math.sqrt(1.0 + abs(x) / RISK)
```

**L97** util(x) = x / √(1 + |x| / RISK). Close to x for small amounts, square-root-like for big ones, and symmetric: util(−x) = −util(x). See section 3.8.

> **SAY IT LIKE THIS**
> "We maximise a utility with the same shape as the score: small profits count fully, big ones only like a square root, so the bot prefers steady wins to gambles."

### Lines 100–116 · The opponent's value as a straight line

```
100 def coefs(st, reads, rule, q, L):
101     """Opponent fair = A0 + A1*t + (B0 + B1*t)*O, before tau / kappa terms.
102
103     st: (bv, hb, npk, pk, N, C0, mo) public state; q: my quote it reads (None
104     for none); L: the clamped implied hand, or None.
105     """
106     bv, hb, npk, pk, N, C0, mo = st
107     cm0 = 5 - npk + hb
108     A0 = bv + pk + cm0 * C0 / N
109     B0 = 1 - cm0 / N
110     if not reads or q is None:
111         return A0, 0.0, B0, 0.0
112     if L is None:
113         c1 = 10 - npk + hb
114         return A0, (q - bv - pk) - c1 * C0 / N, B0, c1 / N
115     c1 = 5 - npk
116     return A0, (L - pk) - c1 * C0 / N, B0, c1 / N
```

st holds the public state as the opponent saw it (built by _state, line 468): bv = board sum, hb = hidden board cards, npk = number of their peeks, pk = estimated sum of my peeked cards, N = cards unknown to them, C0 = total of the cards they have not seen (still including their own hand), mo = the average unknown card if their hand is average.

**L106** Unpack st.

**L107** cm0 = the cards they cannot see that count in V, apart from their own hand: my unpeeked cards (5 − npk) plus the hidden board (hb).

**L108** A0 = visible board + my peeked cards + cm0 cards at the average unknown value C0 / N (the O part of that average goes into B0).

**L109** B0 = 1 − cm0 / N: their value rises by B0 per point of their own hand. It is below 1 because a high hand makes the unknown cards lower on average.

**L110–111** No reading (naive, or I have not quoted yet): fair = A0 + B0·O.

**L112–114** A raw reader adds t × [q − bv − pk − c1 × (C0 − O) / N] with c1 = 10 − npk + hb: my quote minus what a quote from an average hand would be. Written as a straight line: A1 = (q − bv − pk) − c1·C0 / N and B1 = c1 / N.

**L115–116** A reader with a clamped implied hand L (5 or 65) adds t × (L − pk − (5 − npk) × the average unknown card): the clamped hand minus my average hand.

> **SAY IT LIKE THIS**
> "Every opponent style we model values the stock as a straight line in its hand sum O and its trust t. That is what lets us track it with a Kalman filter."

## Lines 119–120 · implied

```
119 def implied(st, q):
120     return q - st[0] - (5 + st[1]) * st[6]
```

**L120** The hand of mine that quote q implies, for an average opponent hand: q − board − (5 + hidden board cards) × the average unknown card. The CLAMP and SANE readers use it to spot impossible quotes.

## Lines 123–139 · pick_q: which of my quotes a reader uses

```
123 def pick_q(st, rule, mq):
124     """Which of my quotes (newest last) a reader of this rule uses, and clamp."""
125     if not mq:
126         return None, None
127     if rule == SANE:
128         for x in reversed(mq):
129             if 12 <= implied(st, x) <= 58:
130                 return x, None
131         return None, None
132     q = mq[-1]
133     if rule == CLAMP:
134         imp = implied(st, q)
135         if imp > 65:
136             return q, 65.0
137         if imp < 5:
138             return q, 5.0
139     return q, None
```

**L125–126** No quote of mine yet: nothing to read.

**L127–131** SANE: the newest quote whose implied hand is between 12 and 58; if there is none, nothing.

**L132** Every other rule reads my newest quote.

**L133–138** CLAMP: if the quote implies more than 65 or less than 5, also return the clamp value.

**L139** RAW and CUR: the newest quote, no clamp.

## Lines 142–154 · ekf: one Kalman step (the readable version)

```
142 def ekf(h, Ho, Ht, d, S, Vy):
143     """Kalman step for hypothesis h: a reading with gradient (Ho, Ht) in (O, t)
144     and predictive variance S came out d above its prediction; Vy is what is
145     left of its variance afterwards (0 when it was seen exactly).
146     Returns the new (mean O, mean t, P00, P01, P11)."""
147     P00, P01, P11 = h[H_P00], h[H_P01], h[H_P11]
148     PHo = P00 * Ho + P01 * Ht
149     PHt = P01 * Ho + P11 * Ht
150     k0 = PHo / S
151     k1 = PHt / S
152     g = S - Vy
153     return (h[H_MO] + k0 * d, h[H_MT] + k1 * d,
154             P00 - g * k0 * k0, P01 - g * k0 * k1, P11 - g * k1 * k1)
```

The belief of a hypothesis is a Gaussian over (O, t) with means (H_MO, H_MT) and covariance P = [[P00, P01], [P01, P11]]. An observation y behaves like Ho·O + Ht·t + noise; S is its predicted variance and d is how far it came out above the prediction.

**L147** Read the covariance.

**L148–149** P·H: how the observation moves together with O and with t.

**L150–151** The Kalman gains k = P·H / S: how much to move each mean per point of surprise.

**L152** g = S − Vy: how much of the observation's uncertainty was removed. For a quote all of it (Vy = 0); for an answer only part of it, because an answer only gives an interval.

**L153–154** New means = old means + k·d; new covariance = P − g·k·$k^T$.

Note: ekf and mix_into are no longer called. ekf_mix (line 175) does both in one step for speed; these two stay as the readable reference.

## Lines 157–172 · mix_into: hedging against outliers

```
157 def mix_into(h, r, new):
158     """Moment-match r*new + (1-r)*old into h."""
159     m0, m1, a, b, c = new
160     o0, o1 = h[H_MO], h[H_MT]
161     n0 = r * m0 + (1 - r) * o0
162     n1 = r * m1 + (1 - r) * o1
163     h[H_P00] = max(0.05, r * (a + m0 * m0) + (1 - r) * (h[H_P00] + o0 * o0) - n0 * n0)
164     h[H_P01] = r * (b + m0 * m1) + (1 - r) * (h[H_P01] + o0 * o1) - n0 * n1
165     h[H_P11] = max(1e-5, r * (c + m1 * m1) + (1 - r) * (h[H_P11] + o1 * o1) - n1 * n1)
166     h[H_MO] = n0
167     h[H_MT] = min(1.15, max(0.0, n1))
168     lim = math.sqrt(h[H_P00] * h[H_P11]) * 0.999
169     if h[H_P01] > lim:
170         h[H_P01] = lim
171     elif h[H_P01] < -lim:
172         h[H_P01] = -lim
```

After the update we are not sure the observation was "explained" (see EPS_Q). With probability r the new belief is right, with 1 − r the old one is. mix_into replaces this mixture of two Gaussians by one Gaussian with the same mean and covariance ("moment matching").

**L161–162** The mixture means.

**L163–165** The mixture covariance = E[x·$x^T$] − mean·$mean^T$. The floors 0.05 and 0.00001 keep the variances positive.

**L166–167** Store the means; the trust t is kept between 0 and 1.15.

**L168–172** Keep |cov(O, t)| below √(P00·P11), so the correlation stays below 1 and the matrix stays valid.

## Lines 175–204 · ekf_mix: the same, fused (the one actually used)

```
175 def ekf_mix(h, r, Ho, Ht, d, S, Vy):
176     """ekf() followed by mix_into(h, r, ...), fused: the same arithmetic in the same order,
177     without building a tuple for every hypothesis and observation."""
178     P00, P01, P11 = h[H_P00], h[H_P01], h[H_P11]
179     PHo = P00 * Ho + P01 * Ht
180     PHt = P01 * Ho + P11 * Ht
181     k0 = PHo / S
182     k1 = PHt / S
183     g = S - Vy
184     o0, o1 = h[H_MO], h[H_MT]
185     m0 = o0 + k0 * d
186     m1 = o1 + k1 * d
187     a = P00 - g * k0 * k0
188     b = P01 - g * k0 * k1
189     c = P11 - g * k1 * k1
190     n0 = r * m0 + (1 - r) * o0
191     n1 = r * m1 + (1 - r) * o1
192     p00 = max(0.05, r * (a + m0 * m0) + (1 - r) * (P00 + o0 * o0) - n0 * n0)
193     p01 = r * (b + m0 * m1) + (1 - r) * (P01 + o0 * o1) - n0 * n1
194     p11 = max(1e-5, r * (c + m1 * m1) + (1 - r) * (P11 + o1 * o1) - n1 * n1)
195     h[H_P00] = p00
196     h[H_P11] = p11
197     h[H_MO] = n0
198     h[H_MT] = min(1.15, max(0.0, n1))
199     lim = math.sqrt(p00 * p11) * 0.999
200     if p01 > lim:
201         p01 = lim
202     elif p01 < -lim:
203         p01 = -lim
204     h[H_P01] = p01
```

The same arithmetic as ekf followed by mix_into, written out in one function so that Python does not build a tuple for every hypothesis and every observation. This was part of marseille's speed-up. Lines 178–189 are ekf; lines 190–204 are mix_into.

**SAY IT LIKE THIS**

"Each hypothesis keeps a Gaussian belief. Every observation moves it with a Kalman step, blended with the old belief by the chance that the observation was not an outlier."

**Your notes**

# 6. The house-reader attack (lines 207–415)

## What this part is for

The arena had three house bots: naive, reader and sharp. The reader and the sharp value the stock with an exact formula that trusts my last quote with weight 0.8. Once we know their hand sum S exactly, we can predict every quote and answer they will make, and plan a sequence of quotes that steers their belief and then trades against it. Since 15:40 the house bots no longer count in the arena score, but this layer is harmless: it switches on only after two of their quotes fit the formula to the cent (no real team triggered it in 400 deals against 45 teams), and it switches off at the first answer that does not fit.

> **WHY / EVIDENCE**
> It was added in nairobi (v02): +7.4 against the house reader and +10.0 against the sharp, with exactly the same decisions as before against every real team.

### Lines 207–221 · Constants of the attack

```
207 # ── exact house-reader attack ──────────────────────────────────────
208 # The house reader and sharp quote fair = S + (1−T)*board + T*P + m*(5 + h − T*(10 + h)) with
209 # T = 0.8 and P = my last quote, rounded to 0.01. Once two of their quotes (one of them reading
210 # a quote of mine) give the same whole hand sum S, they are steered and harvested with an exact
211 # look-ahead (our merge line's attack layer). The first answer that differs from the model
212 # switches the attack off for the rest of the deal.
213 DECK_SUM = 364
214 RD_T = 0.8          # the house reader / sharp trust in my last quote
215 RD_TOL = 0.0051     # their quotes are rounded to 0.01
216 RD_NQ = 2           # consistent quotes needed, at least one of them reading a quote of mine
217 STEER_IN = 1.98     # inside their +-2 band: no trade, their belief moves
218 HARVEST_OUT = 3.02  # outside it: sharp's sizing gives 2 lots at edge >= 3
219 RESP_MARGIN = 0.2   # extra edge wanted when answering an identified reader
220 DEPTH = 4           # quotes of mine explored with every option
221 LIMIT = 10
```

**L208–212** The house formula: fair = S + (1 − T)·board + T·P + m·(5 + h − T·(10 + h)), with T = 0.8, P = my last quote, m = the average unseen card and h = hidden board cards.

**L213** DECK_SUM = 364, the sum of all 52 cards.

**L214** RD_T = 0.8, their trust in my quote.

**L215** RD_TOL = 0.0051: their quotes are rounded to 0.01, so a true fit is within half a cent.

**L216** RD_NQ = 2: two consistent quotes needed, at least one of them reading a quote of mine.

**L217** STEER_IN = 1.98: a quote 1.98 from their value is inside their ±2 band: no trade, but they read my quote and their belief moves.

**L218** HARVEST_OUT = 3.02: just outside the band: the sharp then trades 2 lots.

**L219** RESP_MARGIN = 0.2: when answering an identified reader, take only with 2.2 points of edge (a small safety margin).

**L220** DEPTH = 4: how many of my future quotes the exact search explores with every option.

**L221** LIMIT = 10: the position limit.

### Lines 224–230 · _lots: the sharp's trade size

```
224 def _lots(e):
225     """sharp's sizing for a taker with edge e: signed lots (+ buy)."""
226     a = abs(e)
227     if a <= 2:
228         return 0
229     n = min(2, int(a - 2) + 1)
230     return n if e > 0 else -n
```

**L226–230** For an edge e (their value − my price): no trade within 2; 1 lot when 2 < |e| < 3; 2 lots from 3 up. The sign says buy (+) or sell (−).

### Lines 233–244 · _Reader: its memory

```
233 class _Reader:
234     """Identifies an exact house reader and plans quotes against it."""
235
236     def __init__(self):
237         self.S = None        # hand sum of the current run of consistent quotes
238         self.nq = 0          # consistent quotes in that run
239         self.nread = 0       # ... of them made after a quote of mine
240         self.done = 0        # their quotes processed
241         self.last_round = 0
242         self.peeked = False
243         self.off = False     # an answer did not fit the model
244         self.check = None    # (tape index, expected sign of their qty, my position then)
```

**L237** S: their hand sum in the current run of consistent quotes.

**L238–239** nq: how many quotes are in the run; nread: how many of them read a quote of mine.

**L240** done: how many of their quotes were already processed.

**L241** last_round: the round of the last processed quote (to notice their swaps).

**L242** peeked: they peeked (their formula then changes), so give up.

**L243** off: an answer did not fit; the attack is off for the rest of the deal.

**L244** check: the answer we expect to my last planned quote, so we can verify the model.

### Lines 246–258 · invert: their hand sum from one quote

```
246     @staticmethod
247     def invert(F, B, h, D, N, P):
248         U0 = DECK_SUM - B - D
249         if P is None:
250             c0, c1 = B, 5 + h
251         else:
252             c0, c1 = (1 - RD_T) * B + RD_T * P, 5 + h - RD_T * (10 + h)
253         a = 1 - c1 / N
254         S = round((F - c0 - c1 * U0 / N) / a)
255         if not 5 <= S <= 65:
256             return None
257         pred = S + c0 + c1 * (U0 - S) / N
258         return S if abs(pred - F) <= RD_TOL else None
```

Solve the house formula for S. F = their quote, B = board sum, h = hidden cards, D = discard sum, N = cards unknown to them, P = my last quote (None if I have not quoted).

**L248** U0 = the sum of all cards not on the board and not discarded (their hand + my hand + hidden board + stack).

**L249–252** The formula written as F = S + c0 + c1·(U0 − S) / N. Without a quote of mine c0 = board and c1 = 5 + h; with one, the 0.8 trust terms come in.

**L253–254** Rearranged: S = (F − c0 − c1·U0 / N) / (1 − c1 / N), rounded to a whole number (a hand sum is an integer).

**L255–256** A 5-card hand sum lies between 5 and 65; otherwise this is not the formula.

**L257–258** Recompute the quote from that S and accept it only if it matches to 0.0051.

### Lines 260–283 · observe: is this quote consistent with the earlier ones?

```
260     def observe(self, F, rnd, obs, P):
261         self.done += 1
262         if self.peeked:
263             return
264         B, h = sum(obs.board[:rnd - 1]), 5 - (rnd - 1)
265         ds = [(r, w, k) for r, w, k in obs.discards if r <= rnd]
266         D, N = sum(k for _, _, k in ds), 47 - (rnd - 1) - len(ds)
267         olds = [k for r, w, k in ds if w == "opp" and r > self.last_round]
268         self.last_round = rnd
269         S = self.invert(F, B, h, D, N, P)
270         if S is not None and self.S is not None:
271             if olds:
272                 lo = self.S - olds[0]
273                 ok = lo + 1 <= S <= lo + 13
274             else:
```

```
                ok = S == self.S
            if ok:
                self.S = S
                self.nq += 1
                self.nread += P is not None
                return
        self.S = S
        self.nq = 1 if S is not None else 0
        self.nread = 1 if (S is not None and P is not None) else 0
```

**L261** Count it.

**L262–263** If they peeked, stop: their formula now includes my peeked card.

**L264–266** The public numbers in that round: board sum B, hidden board h, the discards, D and N.

**L267** Their discards since the last quote I processed (a swap changes S).

**L269** S from this quote alone.

**L270–280** If both this quote and the run give a hand sum, they are consistent when they are equal (no swap), or, after they threw card k, when the new sum is between old − k + 1 and old − k + 13 (the new card is 1 to 13). Then the run grows.

**L281–283** Otherwise a new run starts with this quote.

## Lines 285–305 · sync: feed new quotes and verify predictions

```
    def sync(self, obs, pending=None):
        if any(w == "opp" for _, w in obs.peeks):
            self.peeked = True
        quotes, P = [], None
        for r, who, px, q in obs.tape:
            if who == "me":
                P = px
            else:
                quotes.append((px, r, P))
        if pending is not None:
            quotes.append((pending, obs.round, P))
        for px, r, p in quotes[self.done:]:
            self.observe(px, r, obs, p)
        if self.check is not None:
            idx, want, pos_before = self.check
            if idx < len(obs.tape):
                q = obs.tape[idx][3]
                blocked = (want > 0 and pos_before <= -LIMIT) or (want < 0 and pos_before >= LIMIT)
                if not blocked and ((q > 0) != (want > 0) or (q < 0) != (want < 0)):
                    self.off = True
                self.check = None
```

**L286–287** Any peek by the opponent sets peeked.

**L288–293** Walk the tape, remembering my latest quote P; each opponent quote is stored with the quote of mine it would read.

**L294–295** When I am answering, their current quote is not on the tape yet, so add it.

**L296–297** Process only the new ones.

**L298–305** If I made a planned quote, compare their actual answer with the predicted direction. A mismatch that the position limit does not explain switches the attack off.

## Lines 307–310 · active

```
    def active(self, obs):
        if self.off or self.peeked or self.S is None or self.nq < RD_NQ or self.nread < 1:
            return False
        return not any(w == "opp" and r > self.last_round for r, w, k in obs.discards)
```

**L308** On only if: not off, no peek, S known, at least 2 consistent quotes, at least 1 of them reading my quote.

**L310** And they have not swapped since their last quote (S would be unknown again).

### Lines 312–316 · value: V when S is known

```
312     def value(self, obs):
313         n = sum(obs.unseen)
314         tot = sum((r + 1) * c for r, c in enumerate(obs.unseen))
315         rest = (tot - self.S) / max(n - 5, 1)
316         return sum(obs.hand) + sum(obs.board) + self.S + (5 - len(obs.board)) * rest
```

**L313–315** The unseen pool contains their 5 cards (sum S); the rest average (total − S) / (count − 5).

**L316** V = my hand + board + S + (hidden board cards) × that average.

### Lines 318–327 · their_base: their value for a given S and my last quote P

```
318     @staticmethod
319     def their_base(obs, S, P):
320         B = sum(obs.board)
321         h = 5 - len(obs.board)
322         D = sum(k for _, _, k in obs.discards)
323         N = 47 - len(obs.board) - len(obs.discards)
324         m = (DECK_SUM - S - B - D) / N
325         if P is None:
326             return S + B + (5 + h) * m
327         return S + (1 - RD_T) * B + RD_T * P + m * (5 + h - RD_T * (10 + h))
```

**L320–324** Board sum, hidden cards, discards, N, and the average unknown card m from their side.

**L325–326** Without a quote of mine: S + board + (5 + h)·m.

**L327** With one: the 0.8-trust formula.

### Lines 329–340 · events_left: who quotes next, until the end of the deal

```
329     @staticmethod
330     def events_left(obs):
331         """Remaining quote events from this quote of mine on: ('me'/'opp', round)."""
332         first = not any(r == obs.round for r, _, _, _ in obs.tape)
333         ev = []
334         for r in range(obs.round, 6):
335             mine_first = first if (r - obs.round) % 2 == 0 else not first
336             pair = ["me", "opp"] if mine_first else ["opp", "me"]
337             if r == obs.round:
338                 pair = ["me", "opp"] if first else ["me"]
339             ev += [(w, r) for w in pair]
340         return ev
```

**L332** Am I the first maker in this round (no tape entry this round yet)?

**L334–339** List the remaining quotes as ("me" or "opp", round): what is left of this round, then two per round, alternating who starts.

### Lines 342–397 · plan: exact look-ahead against the house bot

```
342     def plan(self, events, pos, Pl, Fnone, A, V):
343         """Best first quote over the remaining events (exact reader model)."""
344         L = LIMIT
345         t = RD_T
346
347         def F(P):
348             return Fnone if P is None else A + t * P
349
350         memo = {}
351
352         def dfs(i, pos, Pl, depth):
353             if i >= len(events):
354                 return 0.0, None
355             key = (i, pos, Pl, depth)
356             got = memo.get(key)
357             if got is not None:
358                 return got
359             got = memo[key] = step(i, pos, Pl, depth)
360             return got
361
362         def step(i, pos, Pl, depth):
363             who, r = events[i]
364             if who == "opp":
```

```
365                 Q = F(Pl)
366                 g = 0.0
367                 if V - Q - 2 > RESP_MARGIN and pos < L:
368                     n = min(2, L - pos)
369                     g, pos = n * (V - Q - 2), pos + n
370                 elif Q - 2 - V > RESP_MARGIN and pos > -L:
371                     n = min(2, L + pos)
372                     g, pos = n * (Q - 2 - V), pos - n
373                 return g + dfs(i + 1, pos, Pl, depth)[0], None
374             Fl = F(Pl)
375             opts = [V]
376             if depth > 0:
377                 opts += [Fl + STEER_IN, Fl - STEER_IN, Fl + HARVEST_OUT, Fl - HARVEST_OUT, 195.0, 0.0]
378             best, bp = -1e18, V
379             for P in opts:
380                 P = round(min(195.0, max(0.0, P)), 2)
381                 n = _lots(Fl - P)                    # + they buy from me
382                 if n > 0:
383                     n = min(n, pos + L)
384                 elif n < 0:
385                     n = max(n, pos - L)
386                 if n > 0:
387                     g = n * (P + 2 - V)
388                 elif n < 0:
389                     g = -n * (V - P + 2)
390                 else:
391                     g = 0.0
392                 v = g + dfs(i + 1, pos - n, P, depth - 1)[0]
393                 if v > best + 1e-9:
394                     best, bp = v, P
395             return best, bp
396 
397         return dfs(0, pos, Pl, DEPTH if len(events) < 7 else DEPTH - 1)   # keeps round 2 fast
```

A depth-first search over the remaining quotes, with memoisation (memo stores the result for each (event, position, my last quote, depth)).

**L347–348** F(P): their value if my last quote is P (A + 0.8·P), or Fnone if I have not quoted.

**L352–360** dfs with its cache.

**L363–373** Their quote equals their value. I buy 2 if V is above their price + 2 + margin, sell 2 if it is below their price − 2 − margin (within limits).

**L374–377** My quote: the options are my true value V (no steering) or, while depth remains, ±1.98 (steer), ±3.02 (harvest), 195 or 0 (extremes).

**L379–394** For each option: their trade size by the sharp's rule, cut by the position limit; my profit at the true value V; plus the best continuation, in which they now read my new quote.

**L397** Depth 4, or 3 when 7 or more events are left (keeps round 2 fast).

## Lines 399–415 · quote and respond against an identified reader

```
399     def quote(self, obs):
400         V = self.value(obs)
401         mine = [px for r, who, px, q in obs.tape if who == "me"]
402         Pl = mine[-1] if mine else None
403         Fnone = self.their_base(obs, self.S, None)
404         A = self.their_base(obs, self.S, 0.0)
405         _, P = self.plan(self.events_left(obs), obs.position, Pl, Fnone, A, V)
406         if abs(P - V) > 1e-9:
407             e = (Fnone if Pl is None else A + RD_T * Pl) - P
408             want = 0 if abs(e) <= 2 else (1 if e > 0 else -1)
409             self.check = (len(obs.tape), want, obs.position)
410         return P
411 
412     def respond(self, obs, price):
413         edge = self.value(obs) - price
414         need = 2.0 + RESP_MARGIN
415         return 2 if edge > need else (-2 if edge < -need else 0)
```

**L400–405** My value, my last quote, their value without and with my quote, then plan.

**L406–409** If the chosen price is not plain V, remember which way we expect them to answer, to check it next time.

**L413–415** Answering them: trade 2 lots when the edge beats 2.2.

**SAY IT LIKE THIS**

"The _Reader class is a special-purpose exact attack on the organisers' house bots. It only switches on when their quotes match the house formula to the cent, and it turns itself off at the first surprise, so against real teams it does nothing."

**Your notes**

# 7. The Bot: inference (lines 418–750)

How the bot turns what it sees into beliefs about the opponent.

### Lines 418–425 · Creating the bot

```
418 class Bot:
419     def __init__(self):
420         self.H = None        # hypotheses
421         self.done = 0        # tape entries consumed
422         self.ndisc = 0       # discards consumed
423         self.extra = -1      # tape slot of their quote consumed early
424         self.first = None    # do I quote first in odd rounds?
425         self.rd = _Reader()  # exact house-reader attack
```

The engine creates a fresh Bot for every deal, so nothing is kept between deals.

**L420** H: the hypotheses (built at the first call, when the first obs is available).

**L421** done: how many tape entries we have already learned from.

**L422** ndisc: how many discards we have already processed.

**L423** extra: the tape slot of their quote that we learned from early (while answering it).

**L424** first: do I quote first in odd rounds (am I seat 0)? Learned at the first call.

**L425** The house-reader attack layer.

### Lines 427–459 · _init: the prior and the 216 hypotheses

```
427     # ── inference ──────────────────────────────────────────
428     def _init(self, obs):
429         u = obs.unseen
430         U = sum(u)
431         S = sum((r + 1) * c for r, c in enumerate(u))
432         m = S / U
433         v = sum(c * (r + 1 - m) ** 2 for r, c in enumerate(u)) / U
434         mu = 5 * m
435         var = 5 * v * (U - 5) / (U - 1)
436         self.obar = mu
437         H = []
438         combos = [(False, RAW, P_NAIVE)] + [(True, r, (1 - P_NAIVE) * p) for r, p in RULE_PRIOR.items()]
439         for reads, rule, pr in combos:
440             for lam, pl in LAM_PRIOR:
441                 for tau, pt in TAU_PRIOR:
442                     if tau and lam < 1:
443                         continue
444                     if lam < 1:
445                         pt = 1.0
446                     for kap, pk in KAP_PRIOR:
447                         for th, pth in TH_PRIOR:
448                             lw = math.log((1 - P_BLEND) * pr * pl * pt * pk * pth)
449                             H.append([reads, rule, lam, tau, kap, mu, T_MEAN if reads else 0.0,
450                                       var, 0.0, T_SD * T_SD if reads else 1e-6, lw, 0.0, th])
451         for lam, pl in BLEND_LAM:
452             for tau, pt in BLEND_TAU:
453                 for kap, pk in KAP_PRIOR:
454                     for sk, ps in BLEND_SK:
455                         for th, pth in TH_PRIOR:
456                             lw = math.log(P_BLEND * pl * pt * pk * ps * pth)
457                             H.append([False, BLD, lam, tau, kap, mu, W_MEAN,
458                                       var, 0.0, W_SD * W_SD, lw, sk, th])
459         self.H = H
```

**L429–432** From my unseen cards: count U, sum S, average m.

**L433** The variance of one unseen card.

**L434–435** Their hand sum O is 5 cards from that pool: mean 5m and variance 5v·(U − 5) / (U − 1) (cards drawn without replacement). At the start this is about 35 ± 8.

**L436** obar = the prior mean of O, our "average hand".

**L438** The families: naive (reads = False) and the four reader rules, with their prior probabilities.

**L440–447** Every combination of lam, tau, kappa and threshold. tau is only allowed for lam = 1 (L442–443), and for lam < 1 the tau factor is 1 (L444–445).

**L448** The log of the prior probability (the product of the factors), times 0.88 (= not a blend quoter).

**L449–450** The new hypothesis: style fields; belief mean O = mu; t = 0.6 for readers (0 for naive); variance of O; no correlation; variance of t (tiny for naive); log-weight; skew 0; threshold.

**L451–458** Blend quoters: every lam, tau, kappa, skew and threshold. For them "t" is the blend weight W (mean 0.45, sd 0.25).

**L459** Store them: 120 + 96 = 216 hypotheses.

### Lines 461–465 · _prune

```
461     def _prune(self):
462         best = max(h[H_LW] for h in self.H)
463         self.H = [h for h in self.H if h[H_LW] > best - PRUNE]
464         for h in self.H:
465             h[H_LW] -= best
```

**L462** The best log-weight.

**L463** Keep hypotheses within PRUNE (6) of the best.

**L464–465** Shift so that the best is 0 (exp() then never underflows).

### Lines 467–487 · _state: the public numbers as the opponent saw them in round r

```
467     @staticmethod
468     def _state(obs, r, obar):
469         """Public state as the opponent saw it in round r."""
470         board = obs.board[:r - 1]
471         nb = len(board)
472         bv = sum(board)
473         nd = 0
474         ds = 0
475         for d in obs.discards:
476             if d[0] <= r:
477                 nd += 1
478                 ds += d[2]
479         npk = 0
480         for p in obs.peeks:
481             if p[1] == "opp" and p[0] <= r:
482                 npk += 1
483         npk = min(npk, 5)
484         pk = npk * sum(obs.hand) / 5.0
485         N = 47 - nb - nd - npk
486         C0 = 364 - bv - ds - pk
487         return (bv, 5 - nb, npk, pk, N, C0, (C0 - obar) / N)
```

**L470–472** The board cards visible in round r (r − 1 of them), their count and sum.

**L473–478** Swaps made up to round r: count nd and sum ds.

**L479–483** How many times the opponent had peeked by round r (at most 5).

**L484** pk: we do not know which of my cards they saw, so each peek counts as the average card of my hand.

**L485** N = 47 − board − discards − peeks: the cards unknown to them (52, minus their own 5, minus what they have seen).

**L486** C0 = 364 − board − discards − peeked: the total of the cards they have not seen, still including their own hand.

**L487** Return (bv, hidden board cards, npk, pk, N, C0, the average unknown card for an average hand).

### Lines 489–501 · _before: my position and Q before slot j

```
489     @staticmethod
490     def _before(tape, j):
491         """My position and my net lots bought at their quotes, before slot j."""
492         pos = 0
493         Q = 0
494         for k in range(j):
495             r, who, price, qty = tape[k]
496             if who == "me":
497                 pos -= qty
498             else:
499                 pos += qty
500                 Q += qty
501         return pos, Q
```

**L494–500** Replay the tape up to slot j. When I was the maker, the taker's qty moves my position the other way (pos −= qty). When they were the maker, I was the taker (pos += qty), and Q counts the net lots I bought at their quotes.

### Lines 503–511 · _ks: the sign swap-expecting bots use

```
503     @staticmethod
504     def _ks(obs, r, pos):
505         """Sign used by swap-expecting bots: my position, if I may still swap."""
506         if r >= 5 or pos == 0:
507             return 0
508         for d in obs.discards:
509             if d[1] == "me" and d[0] <= r:
510                 return 0
511         return 1 if pos > 0 else -1
```

**L506–511** +1 if I am long and can still swap, −1 if short, else 0. It is 0 in round 5 (all swaps happen before round 5's quotes) and after I have swapped. Opponents with kappa > 0 shift their value by kappa × this, expecting me to swap away a bad card.

### Lines 513–529 · _lastq: my last quote, carried forward

```
513     @staticmethod
514     def _lastq(obs, j, r):
515         """My last quote before tape slot j, carried to round r the way a
516         blend quoter carries it (board cards and my swaps since), or None."""
517         tape = obs.tape
518         k = min(j, len(tape)) - 1
519         while k >= 0 and tape[k][1] != "me":
520             k -= 1
521         if k < 0:
522             return None
523         rq, q = tape[k][0], tape[k][2]
524         for c in obs.board[rq - 1:r - 1]:
525             q += CARRY * (c - 7.0)
526         for d in obs.discards:
527             if d[1] == "me" and rq < d[0] <= r:
528                 q += 7.0 - d[2]
529         return q
```

**L518–522** Find my newest quote before slot j.

**L523–525** Each board card shown since adds CARRY × (card − 7).

**L526–528** Each of my swaps since adds 7 − (the card I threw).

**L529** What a blend quoter would now take from my old quote.

### Lines 531–548 · _see, part 1: one observation

```
531     def _see(self, obs, r, j, price, qty, is_quote):
532         tape = obs.tape
533         st = self._state(obs, r, self.obar)
534         pos, Q = self._before(tape, j)
535         ks = self._ks(obs, r, pos)
536         mq = [tape[k][2] for k in range(min(j, len(tape))) if tape[k][1] == "me"]
537         mqc = mq + [price] if not is_quote else mq
538         if not is_quote:
```

```
            can_buy = pos > -10
            can_sell = pos < 10
            if qty > 0:
                kind = 1
            elif qty < 0:
                kind = -1
            else:
                kind = 0
                if not can_buy and not can_sell:
                    return
```

_see is called once per event: their quote (is_quote = True, price = their quote) or their answer to my quote (is_quote = False, price = my quote, qty = their answer).

**L533–535** The public state at that time, my position, Q and ks.

**L536–537** My quotes so far; a CUR reader answering also reads the very price it is answering.

**L538–548** For an answer: kind = +1 (bought), −1 (sold) or 0 (passed). If the position limit blocked both buying and selling, a pass says nothing, so return.

## Lines 549–564 · _see, part 2: per-hypothesis set-up

```
        cof = {}
        lq = self._lastq(obs, j, r) if is_quote else None
        for h in self.H:
            reads, rule, lam = h[H_READS], h[H_RULE], h[H_LAM]
            if is_quote and lam == 0:
                h[H_LW] += math.log(U_Q)
                continue
            key = (reads, rule)
            c = cof.get(key)
            if c is None:
                src = mqc if (rule == CUR and not is_quote) else mq
                q, L = pick_q(st, rule, src)
                c = cof[key] = coefs(st, reads, rule, q, L)
            A0, A1, B0, B1 = c
            mo, mt = h[H_MO], h[H_MT]
            extra = h[H_TAU] * Q + h[H_KAP] * ks
```

**L549** A cache of line coefficients per (reads, rule): all hypotheses of a family share them.

**L550** My carried last quote, for blend quoters.

**L553–555** A lam = 0 hypothesis says "their quotes carry no information": the quote gets the flat density U_Q and the belief does not change.

**L556–561** The coefficients for this family, from the cache or computed with pick_q and coefs.

**L563–564** The belief means; extra = tau·Q + kappa·ks.

## Lines 565–587 · _see, part 3: learning from their quote

```
            if is_quote:
                oe = lam * mo + (1 - lam) * self.obar
                if rule == BLD:
                    F = A0 + B0 * oe + extra
                    if lq is None:
                        pred, Ho, Ht = F, B0 * lam, 0.0
                    else:
                        F += h[H_SK] * pos
                        pred = (1 - mt) * F + mt * lq
                        Ho = (1 - mt) * B0 * lam
                        Ht = lq - F
                else:
                    pred = A0 + A1 * mt + (B0 + B1 * mt) * oe + extra
                    Ho = (B0 + B1 * mt) * lam
                    Ht = A1 + B1 * oe
                S = (h[H_P00] * Ho * Ho + 2 * h[H_P01] * Ho * Ht + h[H_P11] * Ht * Ht
                     + SIG_Q * SIG_Q)
                d = price - pred
                dens = math.exp(-d * d / (2 * S)) * ISQ2PI / math.sqrt(S)
                ev = (1 - EPS_Q) * dens + EPS_Q * U_Q
                rr = (1 - EPS_Q) * dens / ev
                ekf_mix(h, rr, Ho, Ht, d, S, 0.0)
                h[H_LW] += math.log(ev)
```

**L566** oe = lam·O + (1 − lam)·obar: a lam-honest quoter shows only the fraction lam of its hand's distance from average.

**L567–575** Blend quoter: its own value F (plus skew × my position), then quote = (1 − W)·F + W·(my carried last quote). Ho and Ht are the slopes of the predicted quote in O and in W.

**L576–579** Other styles: the predicted quote from the straight line, and its slopes Ho and Ht.

**L580–581** The predicted variance S = $H^T \cdot P \cdot H$ + $SIG_Q^2$.

**L582** d = the surprise.

**L583** dens = the normal density of the surprise.

**L584** The likelihood with outliers: 90% normal, 10% flat.

**L585** rr = the chance that the quote was explained.

**L586** The Kalman step, blended by rr.

**L587** Add the log-likelihood to the weight.

## Lines 588–624 · _see, part 4: learning from their answer to my quote

```
            else:
                th = h[H_TH]
                if kind > 0:
                    lo, hi = price + th, INF
                elif kind < 0:
                    lo, hi = -INF, price - th
                else:
                    lo = price - th if can_sell else -INF
                    hi = price + th if can_buy else INF
                my = A0 + A1 * mt + (B0 + B1 * mt) * mo + extra
                Ho = B0 + B1 * mt
                Ht = A1 + B1 * mo
                S = (h[H_P00] * Ho * Ho + 2 * h[H_P01] * Ho * Ht + h[H_P11] * Ht * Ht
                     + SIG_R * SIG_R)
                sy = math.sqrt(S)
                if lo == -INF:
                    al, pa, Fa, ta = -INF, 0.0, 0.0, 0.0
                else:
                    al = (lo - my) / sy
                    pa, Fa = pdf(al), Phi(al)
                    ta = al * pa
                if hi == INF:
                    pb, Fb, tb = 0.0, 1.0, 0.0
                else:
                    be = (hi - my) / sy
                    pb, Fb = pdf(be), Phi(be)
                    tb = be * pb
                Z = Fb - Fa
                ev = (1 - EPS_R) * Z + EPS_R / 3.0
                h[H_LW] += math.log(ev)
                if Z < 1e-9:
                    continue
                rr = (1 - EPS_R) * Z / ev
                m1 = (pa - pb) / Z
                vy = S * (1 + (ta - tb) / Z - m1 * m1)
                ekf_mix(h, rr, Ho, Ht, sy * m1, S, vy)
        self._prune()
```

**L589** Their threshold th.

**L590–596** The interval their value must be in: bought → above price + th; sold → below price − th; passed → within ±th, except that a side blocked by the position limit is left open.

**L597–602** The predicted value (mean), its slopes, the predicted variance S (plus $SIG_R^2$ for threshold noise) and its sd sy.

**L603–614** The standardised ends of the interval, al and be, with their density and cumulative values (infinite ends handled).

**L615** Z = the probability of this answer.

**L616** The likelihood, allowing 6% random answers (EPS_R / 3 for each kind of answer).

**L617** Update the weight.

**L618–619** If the answer was impossible under this hypothesis, skip the belief update.

**L620–622** rr; m1 = the mean of the truncated normal (in sd units); vy = its variance.

**L623** The Kalman step: the observation "came out" sy·m1 above the prediction, with vy of uncertainty left.

**L624** Prune after every event.

> **SAY IT LIKE THIS**
> "From a quote we learn a number; from an answer we learn an interval. Both go through the same Kalman step, and each hypothesis' weight goes up or down by how likely it found what happened."

## Lines 626–633 · _see_swap: the opponent swapped

```
626     def _see_swap(self, obs, card):
627         u = obs.unseen
628         U = sum(u)
629         m = sum((r + 1) * c for r, c in enumerate(u)) / U
630         v = sum(c * (r + 1 - m) ** 2 for r, c in enumerate(u)) / U
631         for h in self.H:
632             h[H_MO] += m - card
633             h[H_P00] += v
```

**L627–630** The mean m and variance v of a random unseen card.

**L631–633** Their new hand = old hand − thrown card + new card, so in every hypothesis the mean of O moves by m − card and its variance grows by v.

## Lines 635–664 · _sync: process everything new, in order

```
635     def _sync(self, obs, cur=None):
636         if self.H is None:
637             self._init(obs)
638         tape = obs.tape
639         disc = obs.discards
640         while True:
641             nd = disc[self.ndisc] if self.ndisc < len(disc) else None
642             nt = tape[self.done] if self.done < len(tape) else None
643             if nd is None and nt is None:
644                 break
645             if nd is not None and (nt is None or nd[0] <= nt[0]):
646                 if nd[1] == "opp":
647                     self._see_swap(obs, nd[2])
648                 self.ndisc += 1
649                 continue
650             j = self.done
651             self.done += 1
652             if j == self.extra:
653                 continue
654             r, who, price, qty = nt
655             if who == "opp":
656                 if 0.0 < price < MAXP:
657                     self._see(obs, r, j, price, 0, True)
658             else:
659                 self._see(obs, r, j, price, qty, False)
660         if cur is not None:
661             j = len(tape)
662             if 0.0 < cur < MAXP:
663                 self._see(obs, obs.round, j, cur, 0, True)
664             self.extra = j
```

**L636–637** First call: build the hypotheses.

**L640–649** Walk the discards and the tape together. A discard of round r happens before the quotes of round r, so it goes first. Only opponent discards change our beliefs.

**L650–653** Skip a quote we already learned from early.

**L654–659** Their quote (ignoring the extreme prices 0 and 195, which carry no hand information) → quote update. My quote → answer update.

**L660–664** When answering (cur given), learn from their current quote now, and remember its slot so it is not used twice.

### Lines 666–677 · _ev: E[V | O] = α + β·O

```
666     def _ev(self, obs):
667         """E[V | O] = alpha + beta * O."""
668         u = obs.unseen
669         U = sum(u)
670         S = sum((r + 1) * c for r, c in enumerate(u))
671         k = 5 - len(obs.opp_known)
672         kn = sum(obs.opp_known)
673         hb = 5 - len(obs.board)
674         den = U - k
675         alpha = sum(obs.hand) + sum(obs.board) + hb * (S + kn) / den
676         beta = 1 - hb / den
677         return alpha, beta
```

**L668–670** Unseen count and sum (their unknown cards are among them).

**L671–672** k = their cards I have not peeked (always 5 for us); kn = the sum of the peeked ones.

**L673** The hidden board cards.

**L674–676** The hidden board comes from the unseen pool minus their k unknown cards, which gives α and β as in section 3.1.

### Lines 679–689 · _opp_swap (not used any more)

```
679     @staticmethod
680     def _opp_swap(obs):
681         """The swap the opponent still has (live teams swap in round 4-5): when I am short it
682         is long and throws a low card, so V goes up by about m - (a low card); when I am long, down."""
683         if obs.round >= 5 or obs.position == 0 or any(w == "opp" for _, w, _ in obs.discards):
684             return 0.0
685         pool = sorted(r + 1 for r, c in enumerate(obs.unseen) for _ in range(c))
686         n = len(pool)
687         m = sum(pool) / n
688         card = pool[n // 6] if obs.position < 0 else pool[(5 * n) // 6]
689         return OPP_SWAP_K * (m - card) if abs(m - card) > 3 else 0.0
```

An idea from moscow (v05): price in the swap the opponent may still make. palermo (v10) stopped calling it, because live tests showed it cost points against the top teams. The function is still in the file but is never called.

### Lines 691–698 · _mo: our best guess of O

```
691     def _mo(self):
692         tw = 0.0
693         tm = 0.0
694         for h in self.H:
695             w = math.exp(h[H_LW])
696             tw += w
697             tm += w * h[H_MO]
698         return tm / tw
```

The weighted average of O over all hypotheses.

### Lines 700–712 · _p_reader

```
700     def _p_reader(self):
701         """P(reads my quotes with trust >= 0.5), P(... and without a clamp)."""
702         tw = 0.0
703         trd = 0.0
704         trw = 0.0
705         for h in self.H:
706             w = math.exp(h[H_LW])
707             tw += w
708             if h[H_READS] and h[H_MT] >= 0.5 and h[H_LAM] >= LAM_PROBE:
709                 trd += w
710                 if h[H_RULE] in (RAW, CUR):
711                     trw += w
712         return trd / tw, trw / tw
```

The probability that the opponent is a confident, informative reader (trust ≥ 0.5, lam ≥ 0.4), and the probability of that with rule RAW or CUR. Only used for paid manipulation, which is switched off (P_EXT = P_PROBE = 2.0).

## Lines 714–750 · _exact_reader: does the opponent fit the house reader formula?

```
714     def _exact_reader(self, obs):
715         """Trust t if their quotes since their last swap (3 or more) fit the house
716         reader formula exactly (raw reading of my last quote, one trust t, no
717         trade or swap terms), else None."""
718         last_swap = 0
719         for d in obs.discards:
720             if d[1] == "opp" and d[0] > last_swap:
721                 last_swap = d[0]
722         rows = []
723         lastq = None
724         for r, who, price, qty in obs.tape:
725             if who == "me":
726                 lastq = price
727                 continue
728             if r < last_swap or not 0.0 < price < MAXP:
729                 continue
730             rows.append((price,) + coefs(self._state(obs, r, self.obar), True, RAW, lastq, None))
731         if len(rows) < 3:
732             return None
733         best, bt = None, None
734         for i in range(231):
735             t = i * 0.005
736             num = 0.0
737             den = 0.0
738             for q, A0, A1, B0, B1 in rows:
739                 b = B0 + B1 * t
740                 num += (q - A0 - A1 * t) * b
741                 den += b * b
742             O = num / den
743             err = 0.0
744             for q, A0, A1, B0, B1 in rows:
745                 e = abs(q - A0 - A1 * t - (B0 + B1 * t) * O)
746                 if e > err:
747                     err = e
748             if best is None or err < best:
749                 best, bt = err, t
750         return bt if best < EX_TOL else None
```

**L718–721** Their last swap round (quotes before it used another hand).

**L722–730** rows: each of their quotes since then (not extreme), with the straight-line coefficients of a RAW reader of my last quote.

**L731–732** At least 3 are needed.

**L733–749** For every trust t from 0 to 1.15 in steps of 0.005: the best-fitting O (least squares), then the largest misfit. Keep the t with the smallest largest misfit.

**L750** Return t if every quote fits within 0.02, else None. _quote adds extreme prices when t is about 0.8 (the house reader).

**Your notes**

# 8. The Bot: planning and decisions (lines 752–1112)

How the bot chooses a quote, an answer and a move.

### Lines 752–762 · _slots: who quotes in each remaining slot

```
752     # ── planning ──────────────────────────────────────────────
753     @staticmethod
754     def _slots(first, start):
755         """Quote slots start..9 as (I am the maker, round). Each round has two
756         slots; I quote first in odd rounds iff `first`."""
757         out = []
758         for s in range(start, 10):
759             r = s // 2 + 1
760             me_first = (r % 2 == 1) == first
761             out.append(((s % 2 == 0) == me_first, r))
762         return out
```

**L758–761** Slot s (0 to 9) is in round s // 2 + 1. The first slot of a round belongs to the player who starts that round, and I start the odd rounds if first is True.

Example with first = True: (me, 1) (opp, 1) (opp, 2) (me, 2) (me, 3) (opp, 3) (opp, 4) (me, 4) (me, 5) (opp, 5).

### Lines 764–780 · _bel: their value under one hypothesis

```
764     def _bel(self, st, h, O, q, Q, ks):
765         """Opponent fair under hypothesis h (point estimates) reading my quote q."""
766         L = None
767         if q is not None and h[H_READS]:
768             rule = h[H_RULE]
769             if rule == SANE:
770                 if not 12 <= implied(st, q) <= 58:
771                     q = None
772             elif rule == CLAMP:
773                 imp = implied(st, q)
774                 if imp > 65:
775                     L = 65.0
776                 elif imp < 5:
777                     L = 5.0
778         A0, A1, B0, B1 = coefs(st, h[H_READS], h[H_RULE], q, L)
779         t = h[H_MT]
780         return A0 + A1 * t + (B0 + B1 * t) * O + h[H_TAU] * Q + h[H_KAP] * ks, B0 + B1 * t
```

**L767–777** The reading rules: SANE ignores implausible quotes; CLAMP caps the implied hand.

**L778–780** fair = A0 + A1·t + (B0 + B1·t)·O + tau·Q + kappa·ks, returned with its slope in O.

### Lines 782–794 · _roll: set-up

```
782     def _roll(self, st, h, V, pos, Q, qeff, slots, swapped, lq=None):
783         """Deterministic play-out of `slots` under hypothesis h, with simple
784         rules for me (trade against their belief, extreme quote at +-10).
785         Returns (profit relative to value V, final position)."""
786         O = h[H_MO]
787         lam = h[H_LAM]
788         reads = h[H_READS]
789         rule = h[H_RULE]
790         cur_rule = reads and rule == CUR
791         bld = rule == BLD
792         W = h[H_MT]
793         sk = h[H_SK]
794         pnl = 0.0
```

A fast, deterministic play-out of the remaining slots under one hypothesis, used to value what an action does to the rest of the deal. V is the value of the stock under this hypothesis; all profits are measured against it.

## Lines 795–823 · _roll: my quotes

```
        for me, r in slots:
            ks = 0 if (swapped or r >= 5 or pos == 0) else (1 if pos > 0 else -1)
            f, b = self._bel(st, h, 0, qeff, Q, ks)
            if me:
                if pos >= 10:
                    p = MAXP
                elif pos <= -10:
                    p = 0.0
                elif f > V + 1:
                    p = f - 3.05
                elif f < V - 1:
                    p = f + 3.05
                else:
                    p = f
                if cur_rule:
                    f, b = self._bel(st, h, 0, p, Q, ks)
                e = f - p
                th = h[H_TH]
                if e > th:
                    n = min(2, 10 + pos)
                    pos -= n
                    pnl += n * (p + 2 - V)
                elif e < -th:
                    n = min(2, 10 - pos)
                    pos += n
                    pnl += n * (V - p + 2)
                if reads and (rule != SANE or 12 <= implied(st, p) <= 58):
                    qeff = p
                lq = p
```

**L796** ks for this slot.

**L797** Their value f (and its slope b).

**L799–802** At +10 lots I quote 195, at −10 I quote 0: only the trade that helps me is possible, and readers are pulled my way.

**L803–808** If they value the stock more than 1 above V, quote 3.05 below their value so they buy (I sell above V); if more than 1 below, quote 3.05 above so they sell; otherwise quote their value.

**L809–810** A CUR reader re-reads this price.

**L811–820** If their edge beats their threshold, they trade 2 lots (capped by the limit); my profit is counted at V.

**L821–823** Readers now read this quote (SANE ones only if it is plausible).

## Lines 824–838 · _roll: their quotes

```
            elif lam > 0:
                q = f - b * (1 - lam) * (0 - self.obar)
                if bld and lq is not None:
                    q = (1 - W) * (q + sk * pos) + W * lq
                if V - q - 2 > 0:
                    n = min(2, 10 - pos)
                    pos += n
                    Q += n
                    pnl += n * (V - q - 2)
                elif q - 2 - V > 0:
                    n = min(2, 10 + pos)
                    pos -= n
                    Q -= n
                    pnl += n * (q - 2 - V)
        return pnl, pos
```

**L824** lam = 0 quoters are not simulated (their quotes carry nothing).

**L825** Their quote = their value, with only the fraction lam of their hand's distance from average showing.

**L826–827** Blend quoters mix in my last quote (and their skew).

**L828–837** I buy 2 if V − price − 2 > 0 and sell 2 if price − 2 − V > 0; Q keeps track of the lots I bought at their quotes.

**L838** Return (profit relative to V, final position).

### Lines 840–852 · _plan_setup

```
840     def _plan_setup(self, obs, k):
841         st = self._state(obs, obs.round, self.obar)
842         hs = heapq.nlargest(NPLAN, self.H, key=lambda h: h[H_LW])
843         ws = [math.exp(h[H_LW]) for h in hs]
844         tw = sum(ws)
845         ws = [w / tw for w in ws]
846         mq = [e[2] for e in obs.tape if e[1] == "me"]
847         qe = {}
848         for rule in (RAW, CLAMP, SANE, CUR, BLD):
849             q = pick_q(st, rule, mq)[0]
850             qe[rule] = q
851         swapped = any(d[1] == "me" for d in obs.discards)
852         return st, hs, ws, qe, self._slots(self.first, k + 1), swapped
```

**L841** The public state now.

**L842–845** The 6 heaviest hypotheses and their normalised weights.

**L846–850** For each reading rule, which of my quotes a reader would use now.

**L851** Have I swapped?

**L852** The remaining slots after the current one.

### Lines 854–872 · _public (not used) and _plain_fair (the fallback)

```
854     # ── the three methods the engine calls ──────────────────────
855     @staticmethod
856     def _public(obs):
857         """What an average hand on both sides implies, from public cards only."""
858         u = [4] * 13
859         for k in obs.board:
860             u[k - 1] -= 1
861         for _, _, k in obs.discards:
862             u[k - 1] -= 1
863         mp = sum((r + 1) * c for r, c in enumerate(u)) / sum(u)
864         return sum(obs.board) + (15 - len(obs.board)) * mp
865 
866     @staticmethod
867     def _plain_fair(obs):
868         """Card-counting value, used only if something goes wrong."""
869         u = obs.unseen
870         m = sum((r + 1) * c for r, c in enumerate(u)) / max(1, sum(u))
871         hidden = 10 - len(obs.opp_known) - len(obs.board)
872         return sum(obs.hand) + sum(obs.board) + sum(obs.opp_known) + hidden * m
```

**L856–864** _public: the value implied by the public cards alone (moscow's "first quote = public estimate"; no longer called since palermo).

**L867–872** _plain_fair: card counting: my hand + board + peeked cards + every hidden card at the unseen average. Used only if the main code raises an error.

### Lines 874–905 · The three methods the engine calls

```
874     def quote(self, obs):
875         try:
876             q = self._quote(obs)
877         except Exception:
878             q = self._plain_fair(obs)
879         try:
880             self.rd.sync(obs)
881             if self.rd.active(obs):
882                 return self.rd.quote(obs)
883         except Exception:
884             self.rd.off = True
885         return q
886 
887     def respond(self, obs, price):
888         try:
889             a = self._respond(obs, price)
890         except Exception:
891             e = self._plain_fair(obs) - price
892             a = 2 if e > 2 else (-2 if e < -2 else 0)
893         try:
894             self.rd.sync(obs, price)
```

```
            if self.rd.active(obs):
                return self.rd.respond(obs, price)
        except Exception:
            self.rd.off = True
        return a

    def move(self, obs):
        try:
            return self._move(obs)
        except Exception:
            return None
```

**L874–878** quote: try the real thing; on any error use the card-counting value.

**L879–885** Then the house-reader layer may replace it (only when active). If that layer errors, it is switched off.

**L887–899** respond: the same pattern. The fallback trades 2 lots if the card-counting edge is over 2.

**L901–905** move: on an error, do nothing.

> **SAY IT LIKE THIS**
> "Every engine call is wrapped: if anything goes wrong we fall back to a simple card-counting value, so a bug costs a little edge, never a crash."

## Lines 907–924 · _quote, part 1: set-up

```
    def _quote(self, obs):
        self._sync(obs)
        tape = obs.tape
        j = len(tape)
        if self.first is None:
            self.first = (j == 0)
        pos = obs.position
        _, Q = self._before(tape, j)
        alpha, beta = self._ev(obs)
        st, hs, ws, qe, slots, swapped = self._plan_setup(obs, j)
        ks = self._ks(obs, obs.round, pos)
        cur = obs.cash - obs.spent
        nb = min(2, 10 + pos)
        ns = min(2, 10 - pos)
        comps = []
        for w, h in zip(ws, hs):
            V = alpha + beta * h[H_MO]
            comps.append((w, h, V))
```

**L908** Learn from everything new.

**L910–912** j = the tape length. At my first call: if the tape is empty, I am the first maker (seat 0, odd rounds).

**L913–920** Position, Q, α and β, the planning set-up, ks, the money so far (cur), and how many lots they can buy (nb) or sell (ns) at my quote.

**L921–924** For each top hypothesis: its weight, the hypothesis, and V = α + β·O.

## Lines 926–953 · _quote, part 2: their answer model

```
        rd_cache = {}

        def resp_dist(h, p):
            """Mean, sd and cov(O, .)/sd of their fair when answering price p (cached when
            their answer does not read p, which is every hypothesis but the CUR readers)."""
            if not (h[H_READS] and h[H_RULE] == CUR):
                got = rd_cache.get(id(h))
                if got is None:
                    got = rd_cache[id(h)] = resp_dist_raw(h, p)
                return got
            return resp_dist_raw(h, p)

        def resp_dist_raw(h, p):
            q = p if (h[H_READS] and h[H_RULE] == CUR) else qe[h[H_RULE]]
            f, _ = self._bel(st, h, h[H_MO], q, Q, ks)
            L = None
            if q is not None and h[H_READS] and h[H_RULE] == CLAMP:
                imp = implied(st, q)
                L = 65.0 if imp > 65 else (5.0 if imp < 5 else None)
            if q is not None and h[H_READS] and h[H_RULE] == SANE and not 12 <= implied(st, q) <= 58:
```

```
946                 q = None
947             A0, A1, B0, B1 = coefs(st, h[H_READS], h[H_RULE], q, L)
948             Ho = B0 + B1 * h[H_MT]
949             Ht = A1 + B1 * h[H_MO]
950             cO = h[H_P00] * Ho + h[H_P01] * Ht
951             S = cO * Ho + (h[H_P01] * Ho + h[H_P11] * Ht) * Ht + SIG_R * SIG_R
952             sf = math.sqrt(S)
953             return f, sf, cO / sf
```

**L928–936** resp_dist with a per-quote cache: for every hypothesis except CUR readers, the answer model does not depend on my price, so it is computed once (part of marseille's speed-up).

**L939** Which quote of mine they read: the price itself for CUR, otherwise their current reading.

**L940** Their value f.

**L941–947** The reading rules and coefficients.

**L948–953** The slopes; cO = cov(O, their value); S = var(their value) + SIG_R². Returns the mean, the sd and cO / sd.

## Lines 955–961 · _quote, part 3: the immediate expected profit

```
955         def imm(h, V, p):
956             f, sf, c = resp_dist(h, p)
957             th = h[H_TH]
958             zb = (f - p - th) / sf
959             zs = (p - th - f) / sf
960             return (nb * ((p + 2 - V) * Phi(zb) - beta * c * pdf(zb))
961                     + ns * ((V - p + 2) * Phi(zs) - beta * c * pdf(zs))), f
```

**L957–959** The z-scores of "they buy" (value − p > th) and "they sell" (p − value > th).

**L960** nb lots sold at p + 2 with probability Φ(zb), minus the adverse-selection term β·c·φ(zb): they buy exactly when O, and so V, is high.

**L961** The same for selling to me. Returns (expected profit, their value).

> **SAY IT LIKE THIS**
> "The immediate profit of a quote includes adverse selection: the opponent buys exactly when its hand is high, so the formula subtracts β times the covariance times the normal density."

## Lines 963–976 · _quote, part 4: stage 1, the best price for this trade alone

```
963         # the best price for this trade alone (risk-neutral)
964         f0 = [resp_dist(h, comps[0][2])[0] for _, h, _ in comps[:2]]
965         cands = set()
966         for f in f0:
967             for d in DELTAS:
968                 cands.add(f + d)
969                 cands.add(f - d)
970         best, bp = 0.0, f0[0]
971         for p in cands:
972             v = 0.0
973             for w, h, V in comps:
974                 v += w * imm(h, V, p)[0]
975             if v > best:
976                 best, bp = v, p
```

**L964** Their value under the top two hypotheses.

**L965–969** Candidates: those values ± 2, 2.5, 3, 3.5, 4.5 and 6.

**L970–976** Keep the candidate with the highest weighted immediate profit (default: their value, which means no trade).

### Lines 977–994 · _quote, part 5: stage 2 candidates

```
977         # then weigh what each candidate does to the rest of the deal
978         cands = [bp, f0[0], f0[0] + 3.05, f0[0] - 3.05]
979         if len(f0) > 1:
980             cands += [f0[1] + 6.05, f0[1] - 6.05]
981         imp0 = st[0] + (5 + st[1]) * st[6]
982         if pos >= 10:
983             cands += [MAXP, imp0 + 65.0, imp0 + 58.0]
984         elif pos <= -10:
985             cands += [0.0, imp0 + 5.0, imp0 + 12.0]
986         elif obs.round <= 4 and slots and 0.79 <= (self._exact_reader(obs) or 0.0) <= 0.81:
987             cands += [MAXP, 0.0]
988         elif slots and not slots[0][0]:
989             # paid manipulation only when their next quote shows whether it worked
990             p_rd, p_raw = self._p_reader()
991             if p_raw >= P_EXT:
992                 cands += [MAXP, 0.0]
993             if p_rd >= P_PROBE:
994                 cands += [imp0 + 63.0, imp0 + 7.0, imp0 + 55.0, imp0 + 15.0]
```

**L978–980** The stage-1 price, their value, ±3.05 around it, and ±6.05 around the second hypothesis' value.

**L981** imp0 = the price that would imply a hand of 0 for me, so imp0 + x implies a hand of x.

**L982–985** At the limit, extreme prices are free: at +10 the prices 195, "hand 65" and "hand 58"; at −10 the prices 0, "hand 5" and "hand 12".

**L986–987** Against an exact house reader (trust about 0.8), extremes in rounds 1 to 4.

**L988–994** Paid manipulation: switched off, because P_EXT and P_PROBE (2.0) can never be reached.

### Lines 995–1019 · _quote, part 6: score each candidate with the rest of the deal, then nudge

```
995         bestv, bestp = None, bp
996         for p in cands:
997             p = min(MAXP, max(0.0, p))
998             v = 0.0
999             for w, h, V in comps:
1000                x, f = imm(h, V, p)
1001                e = f - p
1002                ps = pos
1003                if e > h[H_TH]:
1004                    ps -= nb
1005                elif e < -h[H_TH]:
1006                    ps += ns
1007                qeff = p if h[H_READS] else None
1008                if h[H_READS] and h[H_RULE] == SANE and not 12 <= implied(st, p) <= 58:
1009                    qeff = qe[SANE]
1010                x += self._roll(st, h, V, ps, Q, qeff, slots, swapped, p)[0]
1011                v += w * util(cur + pos * V + x)
1012            if bestv is None or v > bestv:
1013                bestv, bestp = v, p
1014        bestp = min(MAXP, max(0.0, bestp))
1015        if -10 < pos < 10:
1016            safe = self._safe_quote(obs, alpha + beta * self._mo())
1017            if safe is not None:
1018                return (1 - SAFE_W) * bestp + SAFE_W * safe
1019        return bestp
```

**L996–997** Clamp to 0..195.

**L999–1006** For each hypothesis: the immediate profit, and the position after their likely answer.

**L1007–1009** Readers will read this new price (SANE ones only if it is plausible).

**L1010** Add the play-out of the rest of the deal.

**L1011** The utility of the whole deal: money so far + position × V + future profit.

**L1012–1013** Keep the best.

**L1014** Clamp.

**L1015–1018** Not at the limit: final quote = 70% planner + 30% safe quote.

**L1019** At the limit: the planner's (extreme) price as it is.

**WHY / EVIDENCE**

Top teams quote close to the opponent's last quote and so hide their information. Moving 15% of the way to that "safe quote" (stockholm, v09) gained +0.46 ± 0.23 live; 30% (test u01) another +0.13; using it against every opponent (bogota, v11) another +0.06. Quoting only the safe quote (test j01) lost −0.85: we keep our edge and only borrow part of the hiding.

## Lines 1021–1037 · _safe_quote

```
1021     @staticmethod
1022     def _safe_quote(obs, v):
1023         """Their last quote, carried for board cards and swaps since, plus SAFE_D towards my value v."""
1024         last = None
1025         for r, who, price, qty in obs.tape:
1026             if who == "opp" and 0.0 < price < MAXP:
1027                 last = (r, price)
1028         if last is None:
1029             return None
1030         rq, q = last
1031         for c in obs.board[rq - 1:obs.round - 1]:
1032             q += c - 7.0
1033         for r, who, out in obs.discards:
1034             if r > rq:
1035                 q += 7.0 - out
1036         q += SAFE_D if v > q else -SAFE_D
1037         return min(MAXP, max(0.0, q))
```

**L1024–1029** Their last real quote (not 0 or 195). If there is none, there is no nudge.

**L1030–1032** Carry it for the board cards shown since (each moves it by card − 7).

**L1033–1035** And for the swaps since (7 − the thrown card).

**L1036** Move 1.3 towards my value: inside their ±2 band, so they should not trade on it.

**L1037** Clamp.

## Lines 1039–1069 · _respond: buy, pass or sell?

```
1039     def _respond(self, obs, price):
1040         self._sync(obs, price)
1041         tape = obs.tape
1042         j = len(tape)
1043         if self.first is None:
1044             self.first = (j != 0)
1045         pos = obs.position
1046         _, Q = self._before(tape, j)
1047         alpha, beta = self._ev(obs)
1048         st, hs, ws, qe, slots, swapped = self._plan_setup(obs, j)
1049         cur = obs.cash - obs.spent
1050         lq = self._lastq(obs, j, obs.round)
1051         nbuy = min(2, 10 - pos)
1052         nsell = min(2, 10 + pos)
1053         best, bq = None, 0
1054         for qty in (0, nbuy, -nsell):
1055             v = 0.0
1056             for w, h in zip(ws, hs):
1057                 V = alpha + beta * h[H_MO]
1058                 if qty > 0:
1059                     x = qty * (V - price - 2)
1060                 elif qty < 0:
1061                     x = -qty * (price - 2 - V)
1062                 else:
1063                     x = 0.0
1064                 qeff = qe[h[H_RULE]] if h[H_READS] else None
1065                 x += self._roll(st, h, V, pos + qty, Q + qty, qeff, slots, swapped, lq)[0]
1066                 v += w * util(cur + pos * V + x)
1067             if best is None or v > best + 1e-9:
1068                 best, bq = v, qty
1069         return bq
```

**L1040** Learn from everything, including their current quote.

**L1043–1044** If my first call is an answer at slot 0, they quoted first, so I am not first in odd rounds.

**L1045–1052** Set-up; nbuy and nsell respect the limit.

**L1054** Three options: pass, buy, sell.

**L1057–1063** The profit now at this hypothesis' V (buy at price + 2, sell at price − 2).

**L1064–1065** Plus the play-out of the rest of the deal (tau readers see the lots I traded through Q + qty).

**L1066** The utility of the whole deal.

**L1067–1068** Keep the best; ties go to the earlier option (pass first).

> **SAY IT LIKE THIS**
> "To answer a quote we compare buy, pass and sell with one yardstick: the profit now plus a play-out of the rest of the deal under the six likeliest opponent models, through the utility."

## Lines 1071–1112 · _move: never peek, swap only in round 5

```
1071     def _move(self, obs):
1072         self._sync(obs)
1073         if obs.round < 5 or any(w == "me" for _, w, _ in obs.discards):
1074             return None
1075         pos = obs.position
1076         r = obs.round
1077         # expected final position, from play-outs of the rest of the deal
1078         st = self._state(obs, r, self.obar)
1079         hs = heapq.nlargest(NPLAN, self.H, key=lambda h: h[H_LW])
1080         ws = [math.exp(h[H_LW]) for h in hs]
1081         tw = sum(ws)
1082         alpha, beta = self._ev(obs)
1083         _, Q = self._before(obs.tape, len(obs.tape))
1084         mq = [e[2] for e in obs.tape if e[1] == "me"]
1085         qe = {rule: pick_q(st, rule, mq)[0] for rule in (RAW, CLAMP, SANE, CUR, BLD)}
1086         seats = (self.first,) if self.first is not None else (True, False)
1087         fpos = 0.0
1088         lq = self._lastq(obs, len(obs.tape), r)
1089         for first in seats:
1090             slots = self._slots(first, 2 * (r - 1))
1091             for w, h in zip(ws, hs):
1092                 V = alpha + beta * h[H_MO]
1093                 qeff = qe[h[H_RULE]] if h[H_READS] else None
1094                 fpos += w * self._roll(st, h, V, pos, Q, qeff, slots, False, lq)[1]
1095         fpos /= tw * len(seats)
1096         u = obs.unseen
1097         U = sum(u)
1098         S = sum((rk + 1) * c for rk, c in enumerate(u))
1099         k = 5 - len(obs.opp_known)
1100         eo = self._mo() - sum(obs.opp_known)
1101         mstack = (S - eo) / (U - k)
1102         hand = obs.hand
1103         fpos = 0.5 * (fpos + pos)
1104         if fpos > 0:
1105             i = min(range(5), key=lambda x: hand[x])
1106             gain = fpos * (mstack - hand[i])
1107         else:
1108             i = max(range(5), key=lambda x: hand[x])
1109             gain = fpos * (mstack - hand[i])
1110         if gain > SWAP_TH[r - 1]:
1111             return ("SWAP", i)
1112         return None
```

**L1072** Keep the beliefs current.

**L1073–1074** Do nothing before round 5, or if I have already swapped.

**L1078–1095** The expected final position fpos: play out round 5's two quotes under the top hypotheses (both seatings if unknown).

**L1096–1101** mstack = the expected value of a fresh card: the unseen total minus their expected unknown cards, divided by the remaining count.

**L1102** My hand.

**L1103** pamplona's change: decide on the average of the position I hold now (pos) and the position I expect to end with (fpos). The play-out tends to predict more round-5 trading than really happens, so the average is steadier.

**L1104–1109** Long → throw my lowest card; short → my highest. gain = (that average position) × (mstack − card).

**L1110–1111** Swap if the expected gain is positive (SWAP_TH for round 5 is 0; it was 4 in marseille).

**WHY / EVIDENCE**

Swapping only in round 5 (rio, v03) gained +0.86 ± 0.39 live on real teams, and local tests of swapping from round 4 lost about 0.3. Lowering the round-5 bar from 4 to 0 (manila, v13) gained another +0.10 ± 0.05 live, and deciding on the average of the current and the expected final position (pamplona, v14) another +0.40 ± 0.13, better against 23 of 24 opponents. By round 5 we know which way we are positioned, and the thrown card (which is public) leaves the opponent only two quotes to react.

**WHY / EVIDENCE**

We never peek: a peek costs 3 points for one random card (a fifth of the opponent's hand), while the opponent's quotes and answers already tell us most of O.

**Your notes**

# 9. Every parameter in one table

| Name | Value | Meaning | How it was chosen |
|---|---|---|---|
| MAXP | 195 | highest price and highest V | 13 × 15 |
| P_NAIVE | 0.3 | prior: does not read my quotes | 0.45 → 0.3 in oslo (+0.12 / +0.20 locally); 0.6 lost −0.91 live (aa01); 0.15 lost locally |
| P_BLEND | 0.12 | prior: blend quoter | 0.25 not significant locally |
| BLEND_LAM | 1 / 0.7 / 0.4 / 0.2 | blend quoters: how much hand shows | from replays |
| BLEND_SK | 0 / 0.9 | blend quoters: skew per lot of my position | from replays |
| W_MEAN, W_SD | 0.45, 0.25 | blend weight on my last quote | 0.7 (with P_BLEND 0.3) tested, worse |
| BLEND_TAU | 0 / 1.5 / 3 | blend quoters: move per lot I buy | |
| CARRY | 0.82 | how a shown card moves a carried quote | ≈ 1 − 9/46 (round 2) to 1 − 6/43 (round 5) |
| RULE_PRIOR | .55 / .20 / .08 / .17 | reader rules RAW / CLAMP / SANE / CUR | |
| LAM_PRIOR | 1 / .7 / .4 / 0 | honesty of their quotes (45 / 12 / 8 / 35%) | more weight on lam 0, or "liars", tested worse |
| TH_PRIOR | 2 (75%), 7 (25%) | their take threshold | helsinki +0.86 locally; removing it −0.45 live; 60/40 −0.53 live |
| TAU_PRIOR | 0 / 1.5 / 3 | move per lot I buy (honest quoters) | |
| KAP_PRIOR | 0 (80%), 3.9 (20%) | expects me to swap | |
| T_MEAN, T_SD | 0.6, 0.3 | reader trust prior | |
| SIG_Q | 0.5 | quote noise | 1.0 and 1.5 tested: not clearly better |
| EPS_Q, U_Q | 0.1, 0.02 | unexplained quotes | |
| SIG_R | 1.0 | threshold noise | |
| EPS_R | 0.06 | random answers | |
| PRUNE | 6 | drop hypotheses this far behind | 12 → 6 in marseille (speed, same play) |
| SWAP_TH | 15 / 12 / 9 / 6 / 0 | swap bar by round | only round 5 is used; 4 → 0 in manila (live +0.10 ± 0.05) |
| DELTAS | 2 … 6 | stage-1 price offsets | |
| NPLAN | 6 | hypotheses used to plan | speed |
| P_EXT, P_PROBE | 2.0 | paid manipulation off | stockholm: no probes |
| EX_TOL | 0.02 | exact-reader fit | |
| LAM_PROBE | 0.4 | probes only against informative readers | (probes off) |
| RISK | 60 | utility curvature | 30 and 120 tested |
| SAFE_D | 1.3 | safe-quote step | 2.0 tested live (bb01): a tie |
| SAFE_W | 0.3 | nudge share | 0.15 then 0.3 gained live; 0.4 tied live (mm04: −0.00 ± 0.12) |
| OPP_SWAP_K | 0.6 | not used | moscow idea, removed in palermo |
| DECK_SUM | 364 | sum of the deck | 4 × (1 + … + 13) |
| RD_T | 0.8 | house bots' trust in my quote | house formula |
| RD_TOL | 0.0051 | house quote rounding | half a cent |
| RD_NQ | 2 | quotes needed to identify a house bot | |
| STEER_IN, HARVEST_OUT | 1.98, 3.02 | inside / outside the house bots' band | |

| Name | Value | Meaning | How it was chosen |
|---|---|---|---|
| RESP_MARGIN | 0.2 | extra edge when answering a house bot | |
| DEPTH | 4 | exact search depth | 3 in round 2 for speed |

**Your notes**

# 10. How we got here

## Version history

Every version changed one idea. "Live" is the paired result on the arena against real teams: the newer upload minus the older one, opponent by opponent on the same opponent uploads (± is two standard errors).

| v | Name | What changed | Live result |
|---|---|---|---|
| 00 | tokyo | starting point: Bayesian hypotheses + planner | the best paired live record at the start |
| 01 | berlin | blend quoters with lam 0.2; exact house readers crushed | +0.07 and +0.11 vs tokyo (noise) |
| 02 | nairobi | exact house-reader attack (_Reader) | +7.4 vs house reader, +10.0 vs sharp; same play vs real teams |
| 03 | rio | swap only in round 5 | +0.86 ± 0.39 vs nairobi |
| 04 | denver | first quote = public estimate | inside moscow |
| 05 | moscow | price in the opponent's coming swap | +0.04 ± 0.19 (neutral) |
| 06 | helsinki | take thresholds 2 / 7, P_NAIVE 0.45 | −0.03 ± 0.34 (neutral live, +0.86 locally) |
| 07 | oslo | P_NAIVE 0.3 | inside lisbon |
| 08 | lisbon | defence against double peekers | −0.02 ± 0.26; the defence was dropped |
| 09 | stockholm | 15% nudge, no paid probes | +0.46 ± 0.23 vs lisbon |
| 10 | palermo | 30% nudge; moscow's two ideas undone | +0.43 ± 0.38 vs the 30% test; Imposters +1.91 |
| 11 | bogota | the nudge against every opponent | +0.06 ± 0.08 vs palermo |
| 12 | marseille | same play, 30–40% faster | +0.02 ± 0.07 vs bogota; 47 of 50 fights won |
| 13 | manila | swap in round 5 whenever the expected gain is positive | +0.10 ± 0.05 vs marseille; 45 of 49 fights won |
| 14 | pamplona | round-5 swap decided on the average of current and expected final position | +0.40 ± 0.13 vs manila; better on 23 of 24 |

## Ideas tested live and dropped

| Test | Result |
|---|---|
| j01: quote only the safe quote | −0.85 ± 0.82: gives away our edge |
| aa01: P_NAIVE 0.6 | −0.91 ± 0.64 |
| z02: no take thresholds | −0.45 ± 0.61 |
| bb01: safe-quote step 2.0 | −0.07 ± 0.10 (tie) |
| ll01: opponent-swap pricing back | −0.02 ± 0.20 (tie) |
| mm03: threshold prior 60/40 | −0.53 ± 0.32 |
| mm04: nudge 40% | −0.00 ± 0.12 (tie: 30% stays) |
| rr02: play-out weighted 0.75 when quoting / answering | −0.36 ± 0.33 (14 opponents), −0.11 ± 0.27 (26): rejected, although it was +0.28 locally |

## How we tested

- **Locally**: against imitation bots of each live team, fitted to that team's public replays and calibrated so that our local scores against them match our live scores. One change at a time; a change was kept only if it was more than two standard errors better on two different sets of deals.
- **Live**: the arena plays every upload on the same deals against the same opponent uploads. Comparing two of our uploads opponent by opponent (a "paired" comparison) removes most of the luck. We kept a change only if

it did not lose live, and the biggest steps (the round-5 swap and the nudge) were found and confirmed this way.

- We learned about other teams only from the public arena pages (standings, fight lists, replays). No other team's code was used.

| Your notes |
| --- |
| |
| |
| |
| |

# 11. Questions the organisers may ask

**Q. Explain your bot in one minute.**

The stock is my hand plus the board plus the opponent's hand, so the only real unknown is the opponent's hand sum O. Our bot keeps 216 hypotheses about how the opponent forms its prices (does it read our quotes, how honest are its quotes, how much edge does it need to trade), each with a Gaussian belief about O that a Kalman filter updates from every quote and answer. To act, it scores each candidate price or answer by the expected profit now plus a quick play-out of the rest of the deal under the six likeliest hypotheses, through a square-root-like utility that matches the scoring rule. The final quote is moved 30% towards a safe price near the opponent's last quote, so we give away less information. We never peek, and we swap only in round 5, whenever the swap is expected to gain.

**Q. Why use many hypotheses instead of one opponent model?**

Opponents are very different: naive bots, readers, followers, hiders. Any single model is wrong against most of them. With a weighted set, the data chooses: after a few quotes, the hypotheses that predicted the opponent well carry almost all the weight.

**Q. What is the Kalman filter doing?**

Under each hypothesis the opponent's quote is a straight line in O (and its trust t) plus noise. For a Gaussian belief and a linear noisy reading, the Kalman filter is the exact Bayes update: move the mean by gain × surprise and shrink the variance. One honest quote can pin O down to about ±1.

**Q. How do you learn anything from a pass?**

A pass means the opponent's value was within its threshold of my price, which is an interval. We take the mean and variance of the belief cut to that interval (a truncated normal) and apply the same Kalman step. A buy or a sell gives a one-sided interval.

**Q. What if the opponent does something strange?**

Every observation has a small chance of being "unexplained": 10% for quotes, 6% for answers. One odd move then cannot destroy a good hypothesis, and the belief update is blended by the chance that the move was explained.

**Q. Why 216 hypotheses, and why prune?**

Five families times honesty, trade-learning, swap expectation and threshold, plus the blend quoters, gives 216. Pruning drops those 403 times less likely than the best, which keeps every call fast (median 0.2 ms locally).

**Q. How is a price chosen?**

In two stages. First, the price with the best immediate expected profit (including adverse selection) among offsets from the opponent's value. Then a few candidates are scored together with a play-out of the rest of the deal under the top 6 hypotheses, through the utility. Then the 30% nudge.

**Q. What is adverse selection, and where is it in the code?**

The opponent buys from me exactly when its hand is high, which is when V is high, so a naive expected-profit formula is too optimistic. Line 960 subtracts $\beta \cdot c \cdot \varphi(z)$, the exact correction when the belief is normal.

**Q. Why a concave utility?**

The score is $\text{sign}(p) \cdot \sqrt{|p|}$ per deal pair, so big wins count less than many small ones. A utility with the same shape makes the bot prefer steady profits to big gambles. We kept RISK = 60: 120 was worse and 30 was not clearly better.

**Q. What is the safe quote, and why 30%?**

The safe quote is the opponent's last quote, moved for new board cards and swaps, plus 1.3 towards our value. It sits inside the opponent's ±2 band, so it reveals little and is hard to trade against. Our final quote is 70% planner and 30% safe quote. Live tests: 15% gained +0.46, 30% another +0.13, nudging against every opponent another +0.06, while quoting only the safe quote lost.

**Q. Why do you never peek?**

A peek costs 3 points and shows one random card out of five. The opponent's quotes and answers already tell us most of O, so the information is worth less than its price.

**Q. Why swap only in round 5?**

By round 5 we know which way we are positioned, so we know which card to throw, and the thrown card is public with only two quotes left for the opponent to react. Moving the swap to round 5 (rio) gained +0.86 live. Since manila we swap in round 5 whenever the expected gain is positive (+0.10 live over the old bar of 4), and since pamplona the decision uses the average of the current and the expected final position (+0.40 live).

**Q. What does the _Reader class do? Is it still needed?**

It is an exact attack on the organisers' house bots (reader and sharp), which value the stock with a fixed formula. It switches on only when their quotes match that formula to the cent and off at the first surprise, so against real teams it never switches on. It is harmless, so we kept it.

**Q. What happens if your code throws an error or is slow?**

Every engine method has a try/except: on an error we fall back to a card-counting value (quote), a simple edge rule (answer) or doing nothing (move). marseille (manila's parent) cut the work per call by 30–40%; locally the median call takes about 0.2–0.4 ms and the slowest a few ms.

**Q. Do you keep anything between deals?**

No. The engine creates a fresh Bot for every deal, and everything starts again from the prior.

**Q. Why are P_EXT and P_PROBE equal to 2.0?**

They are probability thresholds, and a probability can never reach 2, so paid manipulation is switched off. It cost money against real teams (one probe quote lost 100 points in a deal in a local test) and turning it off was neutral to positive.

**Q. What is first?**

Whether we are seat 0, which quotes first in odd rounds. We learn it at our first call: if the tape is empty when we are asked to quote, we are first.

**Q. What are Q and tau?**

Q is the net number of lots we bought at the opponent's quotes. Some opponents raise their value when we buy from them (they conclude our hand is high); tau is how much per lot.

**Q. What are kappa and ks?**

Some bots expect a player who is long to swap away a low card, which raises V. kappa is how much such a bot shifts its value, and ks is the direction: the sign of our position while we can still swap.

**Q. What is the difference between lam and t?**

lam is about THEIR quotes: how much of their own hand shows in them (1 fully, 0 not at all). t is about OUR quotes: how much they believe what our quotes say about our hand.

**Q. Why does your quote sometimes go to 0 or 195?**

Only at the position limit. At +10 lots nobody can sell to us, so a quote of 195 can only be traded in our favour (they would buy at 197), and a reader takes it as a sign of a strong hand. At −10 we quote 0 for the same reason.

**Q. What does the play-out (_roll) assume?**

A simple deterministic future at the hypothesis' mean values: we quote 3.05 off their value when it is more than 1 away from V (so they trade our way); they trade when their edge beats their threshold; we take their quotes when V beats them by more than the 2-point spread.

**Q. Why heapq.nlargest?**

To pick the 6 heaviest hypotheses without sorting the whole list; it is faster.

### Q. What does rr mean in _see?

The probability that the observation was explained by the hypothesis rather than being an outlier. It blends the updated belief with the old one.

### Q. What are the weak spots?

Opponents that hide their hand completely and only trade on big edges give the filter little to learn from; there we rely on the nudge and the threshold hypotheses. HAN SOLO, PaperHands and Genz7 were our hardest head-to-head matchups.

### Q. What would you improve with more time?

Detect the opponent's style within a deal and switch strategy per opponent: in hindsight, the best of our versions per opponent would have scored about +0.75 more than the best single version (part of that is luck), but every detector we tried misfired. More speed would also allow more hypotheses in the planner.

| Your notes |
| --- |
| |
| |
| |
| |
| |

# 12. Glossary

| Term | Meaning |
|---|---|
| V | the value of the stock: the sum of both hands and the whole board |
| O | the opponent's hand sum (5 cards) |
| maker / taker | the maker names a price; the taker buys at price + 2, sells at price − 2 or passes |
| lot | one unit of the stock; at most 2 per trade, at most 10 held |
| position | lots held: + long (gains if V is high), − short |
| fair value | what a bot thinks V is worth |
| naive | an opponent that values the stock from its own cards and the public cards only |
| reader | an opponent that also reads what our quotes imply about our hand |
| blend quoter (follower) | an opponent whose quote is partly a copy of our last quote |
| hider / strategic quoter | an opponent whose quotes say nothing about its hand (lam = 0) |
| trust t | how much a reader believes our quotes |
| lam | how much of its own hand an opponent's quotes reveal |
| tau | how much an opponent's value moves per lot we buy from it |
| kappa | how much an opponent's value moves because it expects us to swap |
| take threshold | the edge an opponent needs before it trades (2 or 7 in our model) |
| hypothesis | one possible opponent style, with its own belief and weight |
| prior / posterior | belief before / after seeing the data |
| likelihood | how probable the observed move was under a hypothesis |
| log-weight | log of a hypothesis' weight; adding log-likelihoods is Bayes' rule |
| Kalman filter | the exact Bayes update of a Gaussian belief from a linear noisy reading |
| covariance | how two uncertain numbers move together (here O and t) |
| truncated normal | a normal distribution cut to an interval |
| moment matching | replacing a mixture by one Gaussian with the same mean and variance |
| outlier allowance | the EPS_Q / EPS_R chance that a move is unexplained |
| pruning | dropping hypotheses far less likely than the best |
| play-out (roll-out) | a quick simulation of the rest of the deal |
| utility | the score-like function used to compare outcomes |
| adverse selection | trading with someone who knows more: they trade exactly when it is bad for you |
| safe quote / nudge | the opponent's last quote moved 1.3 towards our value; our quote moves 30% towards it |
| mirrored deal | the same cards played twice with the seats swapped |
| paired comparison | comparing two bots on the same deals and the same opponents, so the luck cancels |

**Your notes**

# Notes

**Your notes**

**Your notes**